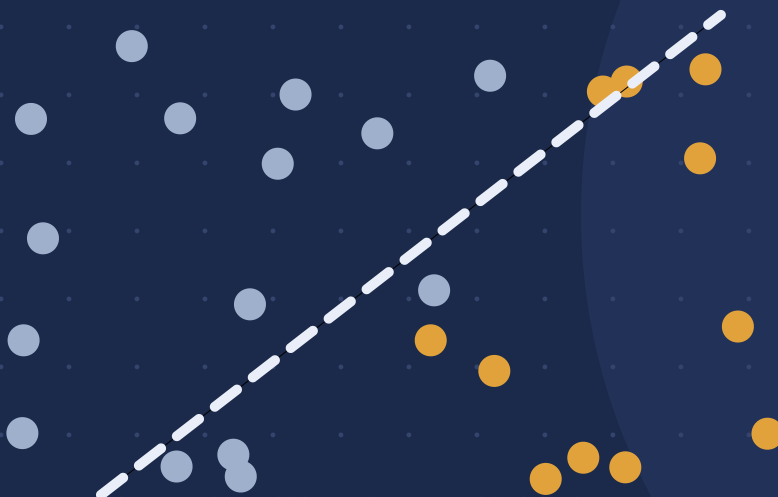


Guessing Which One

A neuron that answers with a probability, a loss that punishes confident mistakes, and learning one example (or one small batch) at a time

Nova can guess how much. Now Nova must choose: is that sound my name, yes or no?

Later: was it 'Nova', a clap, or silence? Asha builds a sigmoid neuron, then a softmax layer, and trains them one example, then one small batch, at a time. Module 4 of the course, slide by slide, in plain words.



What is inside

Book 4 covers Module 4: Linear Neural Networks for Classification. Chapters 1–5: yes/no (binary) classification. Chapters 6–9: choosing among many classes. Chapter 10: tips and the comparison tables. Read them in order.

1	Which one?	3
	Choosing a category is not guessing a number. A straight line cannot do it, so we need a new output.	
2	A neuron that answers with a probability	7
	Keep the weighted sum, squash it with the sigmoid, and the output becomes a probability.	
3	Scoring a guess, and learning one example at a time	12
	Cross-entropy punishes confident mistakes; SGD updates the weights after every single example.	
4	Will Ravi pass? The SGD example by hand	17
	Four students, one feature, two SGD steps done digit by digit, then a check that the curve is moving the right way.	
5	How good is the yes/no machine?	22
	Split the data, count the four kinds of right and wrong, and turn the counts into accuracy, precision, recall and F1.	
6	Many choices: the softmax neuron	26
	One neuron per class, then softmax to turn K scores into K probabilities that add up to one.	
7	Learning in small batches	32
	Not one example, not all of them: a handful at a time. Mini-batch SGD is the standard way to train.	
8	The mini-batch example, by hand	36
	Four points, three classes, one mini-batch of two: every matrix multiplied out, then a trained model used on a new input.	
9	Marking a many-class machine	41
	A KxK confusion matrix, precision and recall per class, two ways to average them, and top-K accuracy.	
10	Tips, debugging, and the big comparison	45
	What to set, what to check when it breaks, and one table that lines up regression, binary and multi-class side by side.	

How to read this book

Each chapter follows the same rhythm: a little story, one big idea, an everyday picture of it, the maths in small steps (first plain, then in matrix form), a worked example with real numbers, and a short check. The coloured boxes tell you what kind of thing you are reading:

- | | |
|--|--|
|  Story — Asha and Nova, the thread through all six books |  Big idea — the one thing to remember |
|  Think of it like... — a real-life picture of the idea |  Stop and think — a question, then Our answer |
|  Maths, step by step — plain maths → matrix form |  Worked example — real numbers, one line per step |
|  On your slides — the exact notation your deck uses |  Careful — common mistakes |
|  Spark — a deeper idea for the curious |  Check yourself — quick questions with answers |

Which one?

Choosing a category is not guessing a number. A straight line cannot do it, so we need a new output.

On your slides: **Module 4, slides 4–10** · What is classification · Examples · Types · Binary classification · Decision boundary · Why not linear regression

STORY

In Book 3, Nova learned to guess *how much*: how loud to reply, and what a flat costs. Tonight Asha plays a sound clip to Nova and asks a different kind of question. "Is that my name? Yes or no?" Nova's screen shows 7.3. Asha frowns. "7.3 what? I did not ask how much. I asked *which one*." Ravi, in the doorway: "Then teach it to choose."

BIG IDEA

A **classification** problem has a fixed list of possible answers, and the model must pick one. Its output should be a *probability* between 0 and 1, not any number at all. That single change, "answer with a probability", is what this whole book is about.

THINK OF IT LIKE...

Sorting post at a hostel counter. Every letter must go into exactly one pigeon-hole: Room 101, Room 102, and so on. You do not say "this letter is 7.3". You say "this one goes to 102", and if the writing is smudged, "probably 102, maybe 107". Classification is sorting into pigeon-holes. Regression was reading a number off a scale.

What classification is

Your slide gives the definition: classification is a **supervised learning** task that predicts a **discrete category** (a class label) from input features. The three key characteristics:

- **Input:** a feature vector $\mathbf{x} \in \mathbb{R}^d$. The same kind of input as in Book 3: d numbers that describe one example.
- **Output:** a class label $y \in \{1, 2, \dots, K\}$. This is *discrete*: it can only be one of K named choices.
- **Goal:** learn a function $f : \mathbb{R}^d \rightarrow \{1, 2, \dots, K\}$ that sends each input to a class.

The difference from regression is the output. Regression predicts a **continuous** value: a price, a temperature, any number on a line. Classification predicts a **category**: spam or not spam, cat or dog.

FOUR EXAMPLES FROM YOUR SLIDES

- **Email spam detection.** Input: the email text, the sender, the attachments. Output: spam or not spam.
- **Medical diagnosis.** Input: symptoms, test results, medical history. Output: disease present or absent.
- **Image recognition.** Input: the pixel values of an image. Output: a category such as cat, dog, or car.

- **Sentiment analysis.** Input: a customer review. Output: positive, negative, or neutral.

Nova's wake-word problem belongs in this list too. Input: an audio clip turned into numbers. Output: "Nova" or "not Nova".

Three types of classification

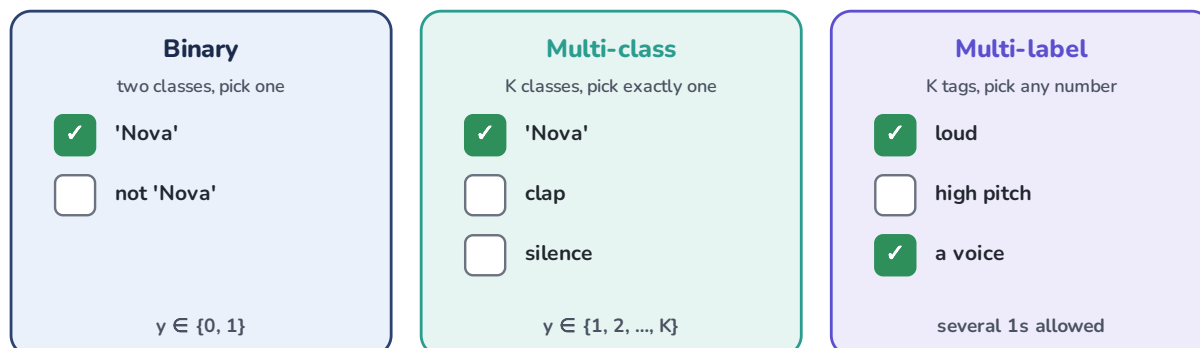


Figure 1.1 The three types, with Nova's versions. Binary: two boxes, tick one. Multi-class: several boxes, tick exactly one. Multi-label: several boxes, tick as many as apply. Notice that only multi-label allows more than one tick.

1. **Binary classification.** Two classes. Your slides write $y \in \{0, 1\}$ or $\{-1, +1\}$. Examples: spam detection, disease diagnosis, fraud detection. This is Chapters 2–5.
2. **Multi-class classification.** $K > 2$ classes, $y \in \{1, 2, \dots, K\}$. Each example belongs to *exactly one* class. Examples: digit recognition (0–9), animal classification. This is Chapters 6–9.
3. **Multi-label classification.** Each example can belong to *several* classes at the same time. Examples: a film that is action *and* comedy, tagging a document with many topics. Your deck names it and moves on; so will we.

! CAREFUL

Multi-class and multi-label are not the same. "Cat, dog, or car?" is multi-class: one answer. "Which tags fit: outdoor, animal, night?" is multi-label: any number of answers. In this book, "many classes" always means multi-class.

Binary classification, written properly

We start with the simplest case, two classes, because everything else is built on it. Your slide fixes the notation.

PLAIN MATHS, ONE EXAMPLE

Binary classification assigns each input to one of two classes, $y \in \{0, 1\}$.

- $y = 1$: the **positive class**, the thing we are hunting for (spam, disease present, "Nova"). "Positive" does not mean "good".
- $y = 0$: the **negative class**, everything else.

The model outputs the **probability of class 1**, $f(\mathbf{x})$, a number like 0.93. Then decide: $\hat{y} = 1$ if $f(\mathbf{x}) \geq 0.5$, else $\hat{y} = 0$.

ALL N EXAMPLES

The dataset is N pairs of (features, label):

$$D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$$

Stack the inputs as rows of $\mathbf{X}$ ($N \times d$) and the labels as a column $\mathbf{y}$ of N bits. The model gives a column of N probabilities; compare each with 0.5 to get N decisions. Chapter 2 fills in what f is.



The picture: a decision boundary

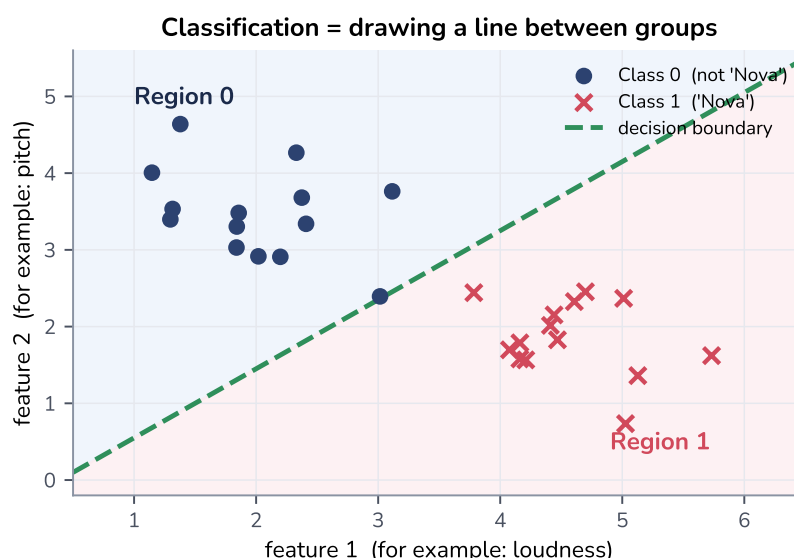


Figure 1.2 Your slide 9, redrawn. Each dot or cross is one example, placed by its two features. The dashed green line is the decision boundary: on one side the model says 0, on the other side 1. Notice that the boundary is a straight line. Everything in this book draws straight boundaries.

With two features you can draw every example as a point. The model's job is to draw a line that separates the two groups. The line is the decision boundary. The two sides are **Region 0** and **Region 1**. A new point is classified by one question: which side of the line is it on? With d features the "line" becomes a flat sheet in d dimensions, a **hyperplane**. You cannot draw it, but the idea is the same.

Why not linear regression?

Ravi asks: "Nova already has a neuron from Book 3 that outputs a number. Why not use it, and call anything above 0.5 a yes?" Your slide 10 tests exactly that, and it fails. The Book 3 neuron computes $\hat{y} = wx + b$, a straight line. It can output any number: $\hat{y} \in (-\infty, \infty)$. But we need a probability: $\hat{y} \in$

$[0, 1]$. A line will happily predict -0.5 for one student and 1.5 for another. "Minus 50% chance of passing" means nothing.

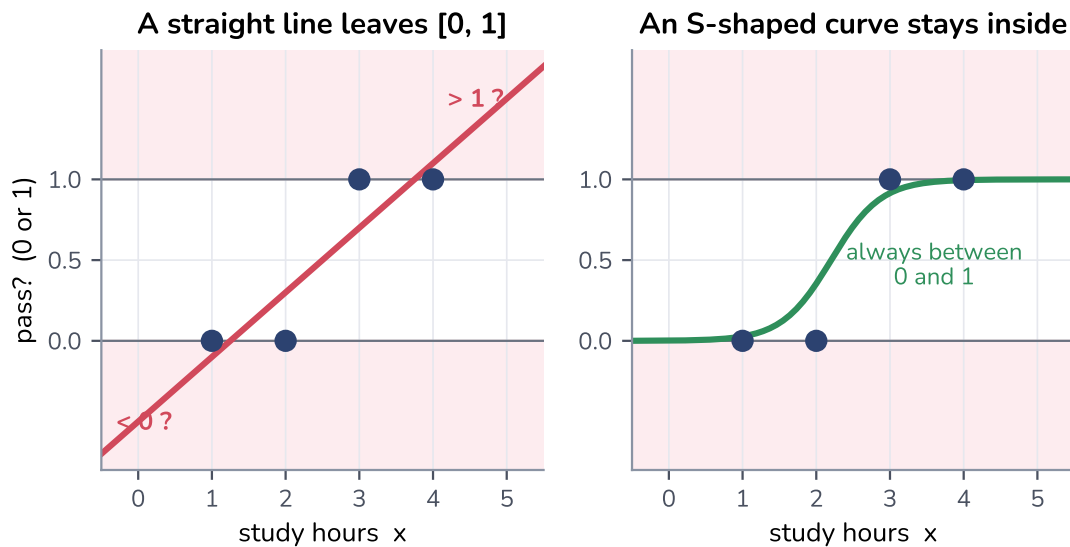


Figure 1.3 Left: the study-hours data of Chapter 4 (1, 2 hours $\rightarrow$ fail; 3, 4 hours $\rightarrow$ pass) with the best straight line. The line leaves the allowed band: below 0 on the left, above 1 on the right. Right: an S-shaped curve through the same points stays inside the band, however far you go. This curve is the fix.

Your slide states the solution in one line: *use a function that maps $(-\infty, \infty) \rightarrow [0, 1]$* . Take the straight line's output, whatever it is, and squash it into the band between 0 and 1. The squashing function is called the **sigmoid**, and it is the subject of Chapter 2.

ON YOUR SLIDES

Slide 4: the definition, with $f: \mathbb{R}^d \rightarrow \{1, \dots, K\}$. Slide 5: the four examples. Slide 6: the three types. Slide 8: binary notation $y \in \{0, 1\}$, positive and negative class, dataset D , decision rule at 0.5. Slide 9: the two-region picture. Slide 10: linear regression outputs $(-\infty, \infty)$, we need $[0, 1]$, solution: a function that maps one to the other.

CHECK YOURSELF

1. What is the difference between regression and classification? **Regression outputs a continuous number; classification outputs one of a fixed set of classes.**
2. "Is this email spam?" is which type? **Binary classification, with spam as the positive class.**
3. Why is a plain straight line a bad classifier? **Its outputs can be below 0 or above 1, which cannot be probabilities.**

IN ONE BREATH

- Classification = pick a class from a list: binary (2), multi-class (K , one answer), multi-label (many).
- Binary: $y \in \{0, 1\}$, class 1 is "positive". The model outputs $P(y = 1)$; predict 1 if it is ≥ 0.5 .
- The decision boundary is a straight line (a hyperplane) between the two regions.
- A straight line outputs any number. We need something that squashes it into $[0, 1]$.

A neuron that answers with a probability

Keep the weighted sum, squash it with the sigmoid, and the output becomes a probability.

On your slides: **Module 4, slides 12–20** · Logistic regression · The sigmoid function · Properties of sigmoid · The four components · Matrix form of data · Single neuron · Model prediction · Making predictions

STORY

Asha keeps the neuron from Book 3. It still adds up the inputs with weights. But she bolts a new part onto its output: a small curve that takes any number, however big or small, and bends it into the space between 0 and 1. She plays the clip again. Nova's screen shows 0.93. "Ninety-three percent sure it is my name," says Asha. Ravi: "And if it shows 0.5?" Asha: "Then it has no idea."

BIG IDEA

A binary classifier is one neuron in two steps. Step 1: the weighted sum $z = \mathbf{w}^\top \mathbf{x}$, exactly as in regression. Step 2: squash z with the **sigmoid** $\sigma(z) = \frac{1}{1+e^{-z}}$. The result $\hat{y} = \sigma(z)$ is the probability that the input belongs to class 1. Everything else in the chapter is about the shape of that squashing curve.

THINK OF IT LIKE...

A dimmer knob for a lamp. You can turn the knob as far as you like in either direction (that is z , any number). But the lamp can only go from fully off (0) to fully on (1). Turn far left, the lamp is off and turning more changes nothing. Turn far right, it is fully on. In the middle, small turns make a big difference. The sigmoid is the dimmer: unlimited knob in, brightness between 0 and 1 out.

Logistic regression

Your slide's definition: **logistic regression** is a linear model for binary classification that uses the sigmoid function to output probabilities. Do not be confused by the name. It has "regression" in it for historical reasons, but it is a classifier.

The key idea has three lines:

1. Compute a linear combination: $z = \mathbf{w}^\top \mathbf{x} + w_0$. This is the weighted sum you know.
2. Apply the sigmoid: $\hat{y} = \sigma(z)$.
3. Read the output as a probability: $\hat{y} = P(y = 1 \mid \mathbf{x}) \in [0, 1]$. The symbol $P(y = 1 \mid \mathbf{x})$ is read "the probability that y is 1, given the input $\mathbf{x}$ ".

Your deck folds the bias w_0 into $\mathbf{w}$ by putting a 1 in front of $\mathbf{x}$, just as Book 3 did. Then the whole model is one formula, equation (1) on your slide:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

The sigmoid function

Here is the curve. The symbol σ is the Greek letter sigma, and $e \approx 2.718$ is the usual constant. The sigmoid is also called the **logistic function**.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

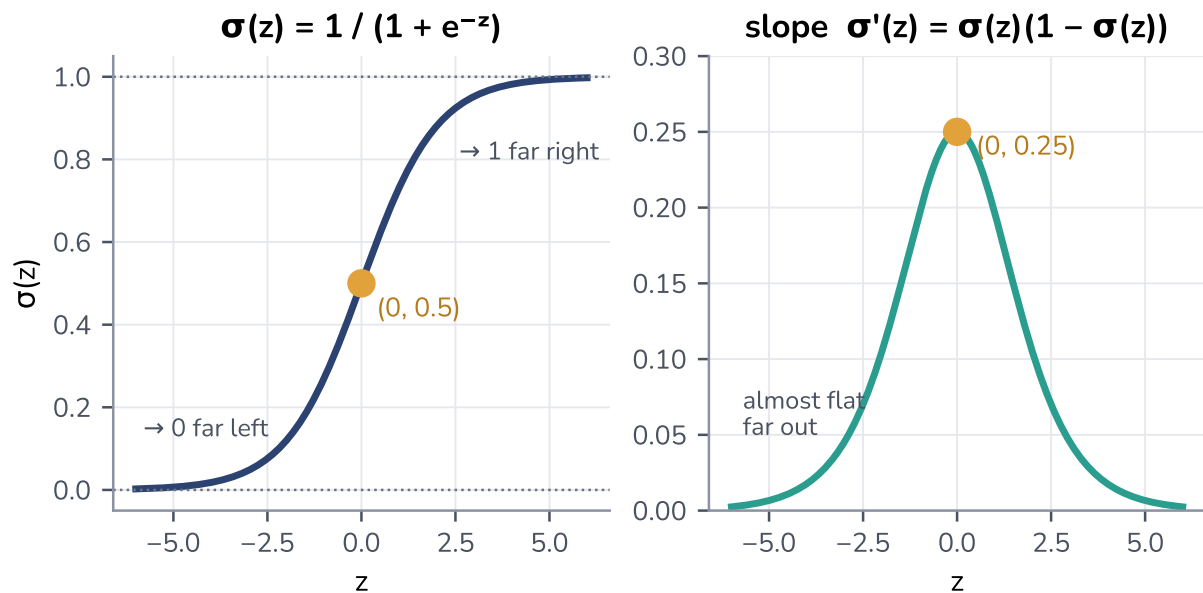


Figure 2.1 Left: the sigmoid. It passes through $(0, 0.5)$, flattens towards 0 on the left and towards 1 on the right. Right: its slope $\sigma'(z)$. Notice the slope is biggest in the middle, exactly 0.25 at $z = 0$, and almost zero far out on either side. Where the curve is flat, learning is slow.

Let us check the three landmarks by hand. At $z = 0$: $e^0 = 1$, so $\sigma(0) = \frac{1}{1+1} = 0.5$. For a big positive z : e^{-z} is tiny, so $\sigma(z) \approx \frac{1}{1+0} = 1$. For a big negative z : e^{-z} is huge, so $\sigma(z) \approx \frac{1}{\text{huge}} = 0$.

THE DERIVATIVE

The slope of the sigmoid has a lovely form. Your slide gives it without proof:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

You do not need a new formula to find the slope. You only need the value of $\sigma(z)$ itself. At $z = 0$: $\sigma'(0) = 0.5 \times (1 - 0.5) = 0.25$. This is the peak of the right-hand plot. Your slide says the derivative property "makes sigmoid ideal for gradient-based learning": the gradient is cheap to compute, because the forward pass already gave us $\sigma(z)$.

PROPERTIES, FROM YOUR SLIDE

Range

$\sigma(z) \in (0, 1)$. Never exactly 0 or 1, always strictly between. Perfect for probabilities.

Monotonic

Always increasing. Bigger z always means bigger $\hat{y}$. The order of inputs is never flipped.

Smooth

Differentiable everywhere. No corners, no jumps. Gradient descent can always find a slope.

Symmetry

$\sigma(-z) = 1 - \sigma(z)$. The curve on the left is the mirror of the curve on the right, flipped upside down.

And the behaviour at the ends: when $z \rightarrow +\infty, \sigma(z) \rightarrow 1$ (confident class 1). When $z \rightarrow -\infty, \sigma(z) \rightarrow 0$ (confident class 0). When $z = 0, \sigma(z) = 0.5$ (uncertain).

The four components, again

Every learning problem in this series has the same four parts. Here they are for binary classification, straight from your slide 16.

1. Data

d -dimensional input vectors and binary labels $y \in \{0, 1\}$.

2. Model

A single neuron with sigmoid activation.

3. Objective

Binary cross-entropy loss (Chapter 3).

4. Learning

Stochastic gradient descent, SGD (Chapter 3).

Your slide adds a note: the deck uses *stochastic* gradient descent instead of Book 3's batch version "for efficiency", and says this is "for learning purposes only". In other words, the deck wants to teach you SGD here. Batch GD would work too.

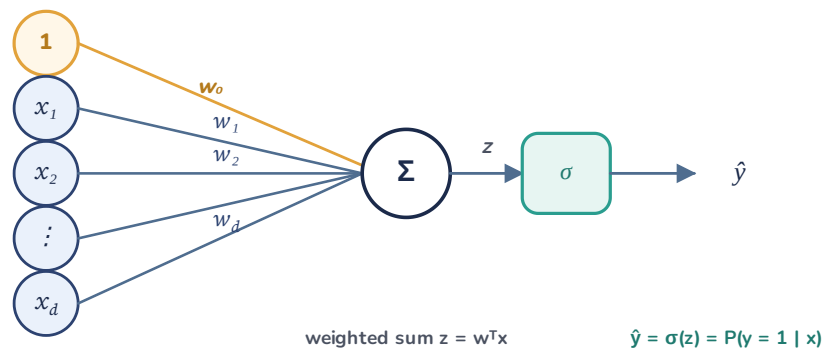


Figure 2.2 Logistic regression as one neuron. Inputs on the left, weights on the arrows, the bias enters through the constant input 1. The Σ node adds everything into z . The new part is the σ box, which squashes z into $\hat{y}$. In Book 3 that box was the identity $f(z) = z$. Notice: swapping one box turns a regressor into a classifier.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One example with d features, and a 1 in front for the bias:

$$\mathbf{x}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^\top$$

Its label is a single bit: $y^{(i)} \in \{0, 1\}$.

The weights are $d + 1$ numbers, w_0 being the bias:

$$\mathbf{w} = [w_0, w_1, \dots, w_d]^\top$$

The prediction for this one example:

$$z^{(i)} = w_0 + w_1 x_1^{(i)} + \dots + w_d x_d^{(i)}, \quad \hat{y}^{(i)} = \sigma(z^{(i)})$$

MATRIX FORM

Stack the N examples as rows of the **design matrix**, with a column of ones first:

$$\mathbf{X} \in \mathbb{R}^{N \times (d+1)}, \quad \mathbf{y} \in \{0, 1\}^N, \quad \mathbf{w} \in \mathbb{R}^{d+1}$$

$\mathbf{X}$ is $N \times (d + 1)$ and $\mathbf{w}$ is $(d + 1) \times 1$, so $\mathbf{X}\mathbf{w}$ is $N \times 1$: one logit per row. Apply σ to every entry:

$$\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w})$$

One multiplication, all N predictions. That is what the computer runs. Your slide's key point: the only difference from regression is that $y^{(i)}$ is discrete, 0 or 1.

Model prediction

Your slide 19 writes the prediction in three numbered equations. First the weighted sum:

$$z = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \quad (2)$$

Then the squash:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

And the meaning: $\hat{y} = P(y = 1 \mid \mathbf{x}) \in [0, 1]$, equation (4). The number z has a name: it is the **logit**, also called the **pre-activation**, because it is the value *before* the activation function is applied. $\hat{y}$ is the **predicted probability**.

From probability to decision

A probability is not yet a choice. Your slide 20 gives the **decision rule**:

$$\text{predicted class} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases} \quad (5)$$

Now a small but important trick. Since $\sigma(z) = 0.5$ exactly when $z = 0$, and σ is always increasing, " $\hat{y} \geq 0.5$ " is the same as " $z \geq 0$ ". So we can decide without computing the sigmoid at all:

$$\text{predicted class} = \begin{cases} 1 & \text{if } \mathbf{w}^\top \mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^\top \mathbf{x} < 0 \end{cases} \quad (6)$$

The 0.5 rule is the same as the 'z = 0' rule

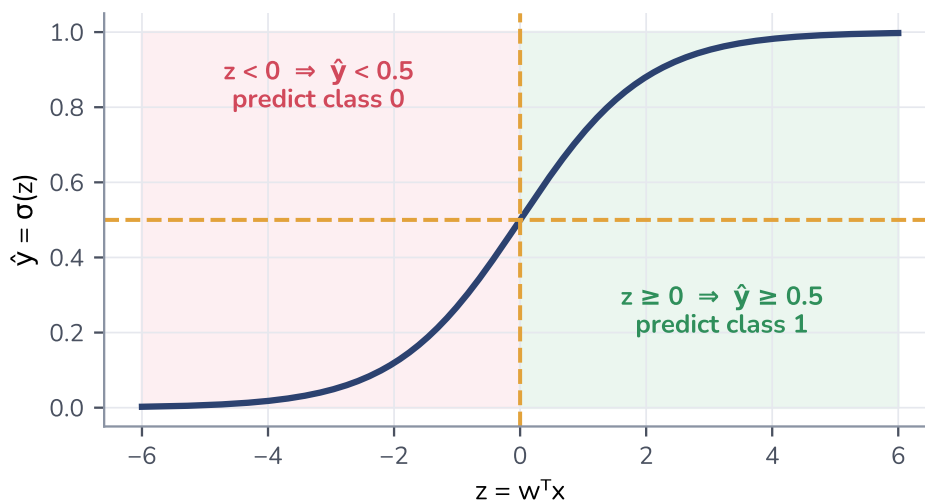


Figure 2.3 The two rules are one rule. The dashed horizontal line is $\hat{y} = 0.5$; the dashed vertical line is $z = 0$. They cross exactly on the curve. Left of the line: predict 0. Right of it: predict 1.

The set of inputs where $\mathbf{w}^\top \mathbf{x} = 0$ is the **decision boundary**. It is a straight line in two dimensions, a flat sheet in more: a **hyperplane**. That is the dashed green line in Figure 1.2. The weights $\mathbf{w}$ decide where it sits and how it tilts.

? STOP AND THINK

Ravi asks: "If we only need the sign of z to decide, why bother with the sigmoid at all? The perceptron in Book 2 did exactly that."

✓ OUR ANSWER

Two reasons. First, the probability is useful on its own: 0.51 and 0.99 are both "class 1", but you would trust them very differently. Second, and this is the real reason, learning needs a smooth curve. The perceptron's step function has no slope, so gradient descent cannot use it. The sigmoid has a slope everywhere, so we can learn $\mathbf{w}$ by going downhill. The next chapter does exactly that.

📖 ON YOUR SLIDES

Slide 12 defines logistic regression and writes $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x}) = 1/(1 + e^{-\mathbf{w}^\top \mathbf{x}})$ as equation (1). Slide 13 plots $\sigma(z)$ and $\sigma'(z)$, marking (0, 0.5) and (0, 0.25). Slide 14 lists the properties (range, monotonic, smooth, symmetry, derivative) and the behaviour at $\pm\infty$. Slide 16 is the four components with the SGD note. Slide 17 is the matrix form: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$, $\mathbf{y} \in \{0, 1\}^N$, $\mathbf{w} \in \mathbb{R}^{d+1}$. Slide 18 is the neuron drawing with Σ and σ . Slide 19 is equations (2)–(4) and the word *logit*. Slide 20 is the decision rule, equations (5) and (6), and "the decision boundary is $\mathbf{w}^\top \mathbf{x} = 0$, a hyperplane".

! CAREFUL

$\hat{y}$ is a probability, not a class. The class comes *after* the 0.5 rule. The loss (next chapter) always uses the probability $\hat{y}$, never the rounded class. Rounding throws away the information that learning needs.

✓ CHECK YOURSELF

1. What does the sigmoid do to a number? **Squashes any real number into the interval (0, 1).**
2. What is $\sigma(0)$? And $\sigma'(0)$? **0.5 and 0.25.**
3. What is a logit? **The weighted sum $z = \mathbf{w}^\top \mathbf{x}$, the value before the activation.**
4. The decision boundary is the set of points where...? **$\mathbf{w}^\top \mathbf{x} = 0$, equivalently $\hat{y} = 0.5$.**

🧠 IN ONE BREATH

- Logistic regression = weighted sum $z = \mathbf{w}^\top \mathbf{x}$, then sigmoid $\hat{y} = \sigma(z) = 1/(1 + e^{-z})$.
- $\hat{y}$ is $P(y = 1 \mid \mathbf{x})$. Sigmoid: range (0, 1), increasing, smooth, $\sigma(-z) = 1 - \sigma(z)$, $\sigma' = \sigma(1 - \sigma)$.
- Matrix form: $\hat{\mathbf{y}} = \sigma(\mathbf{X}\mathbf{w})$, all predictions in one multiplication.
- Decide: class 1 if $\hat{y} \geq 0.5$, which is the same as $z \geq 0$. The boundary $\mathbf{w}^\top \mathbf{x} = 0$ is a hyperplane.

Scoring a guess, and learning one example at a time

Cross-entropy punishes confident mistakes; SGD updates the weights after every single example.

On your slides: **Module 4, slides 21–29** · Binary cross-entropy · Total loss · Why cross-entropy · From batch to SGD · SGD · The gradient · SGD algorithm · SGD vs batch GD · Benefits and challenges

STORY

Nova now says 0.93 for Asha's name. But for a slammed door it also says 0.88. "Confidently wrong," says Ravi. "How do we punish that?" Squared error from Book 3 would charge only $0.88^2 = 0.77$. Not much for being 88% sure of nonsense. Asha needs a loss that gets angry fast. And she has ten thousand clips: waiting for all of them before every tiny step is too slow.

BIG IDEA

Two ideas. (1) **Cross-entropy**: the loss is $-\log$ of the probability the model gave to the *true* class. Small if it was nearly right, huge if it was confidently wrong. (2) **Stochastic gradient descent**: do not wait for all N examples. Pick one at random, compute its gradient, take a step, repeat. The gradient is $(\hat{y} - y) \mathbf{x}$: error times input, exactly like regression.

THINK OF IT LIKE...

A forecaster who says "90% chance of rain" and it stays dry loses far more trust than one who said "55%". Cross-entropy is that trust penalty: mild for a shrug, huge for a confident miss. And SGD is tasting the curry one spoon at a time and adjusting the salt after each spoon, instead of serving the whole pot, collecting every complaint, and only then adding salt.

Binary cross-entropy loss

Your slide is blunt: "We cannot use squared error for classification. Instead, we use cross-entropy loss." For a single example $(\mathbf{x}^{(i)}, y^{(i)})$ with prediction $\hat{y}^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)})$, the loss is equation (7):

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = - \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

It looks like two terms, but only one of them is ever switched on, because y is 0 or 1:

- If $y^{(i)} = 1$: the second term is multiplied by 0 and vanishes. Loss = $-\log(\hat{y}^{(i)})$. It punishes $\hat{y}$ for being far from 1.
- If $y^{(i)} = 0$: the first term vanishes. Loss = $-\log(1 - \hat{y}^{(i)})$. It punishes $\hat{y}$ for being far from 0.

So the formula is a neat way to write "take $-\log$ of the probability you gave to the right answer". $\log$ is the natural logarithm; $-\log$ of a number between 0 and 1 is always positive.

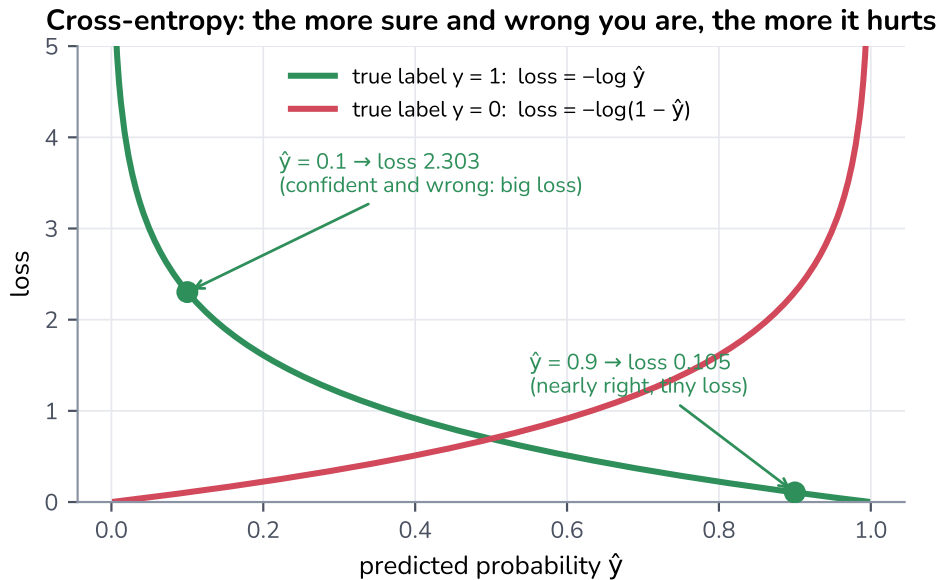


Figure 3.1 Your slide 23. Green: the loss when the truth is 1; red: when the truth is 0. Follow the green curve from right to left: at $\hat{y} = 0.9$ the loss is only 0.105, at $\hat{y} = 0.1$ it is 2.303. Notice that it shoots up towards infinity near the wrong end. Being confidently wrong costs almost everything.

THE TOTAL LOSS

Average the per-example loss over the whole dataset, equation (8):

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

The goal, equation (9), is the same as always: find the weights that make J smallest, $\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w})$.

WHY CROSS-ENTROPY? (YOUR SLIDE 23)

- **Probabilistic interpretation.** It comes from *maximum likelihood estimation*: choose the weights that make the training labels as probable as possible. $-\log$ of a product of probabilities is a sum of $-\log$ terms, and that sum is cross-entropy.
- **Right for probabilities.** It heavily penalises confident wrong predictions. Squared error would only charge $0.88^2 = 0.77$ for Ravi's door; cross-entropy charges $-\log(1 - 0.88) = 2.12$.
- **Convex.** For logistic regression the loss surface is one bowl with a single global minimum. Gradient descent cannot get stuck in a wrong dip.
- **Well-behaved gradients.** It works well with the sigmoid: as you will see below, the gradient comes out clean and simple.

From batch to stochastic gradient descent

Book 3 used **batch gradient descent**. Recap from your slide 24: use *all* N examples for each update, $\nabla J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$, then step $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$, where η (eta) is the learning rate.

Three problems with batch GD:

- **Slow for large datasets:** every single update must touch all N examples.
- **High memory:** all the data must be loaded at once.
- **Redundant work:** when many examples look alike, their gradients say the same thing. Adding them all up is wasted effort.

The solution is **stochastic gradient descent (SGD)**: update the weights using *one random example at a time*. Much faster iterations, much less memory. "Stochastic" just means "involving randomness", because the example is chosen at random.

THE SGD UPDATE RULE

At iteration t :

1. Randomly pick one example $(\mathbf{x}^{(i)}, y^{(i)})$.
2. Compute the gradient for this example only: $\nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$. This is a *noisy estimate* of the true gradient ∇J .
3. Update: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \ell(\mathbf{w}^{(t)}; \mathbf{x}^{(i)}, y^{(i)})$.

Your slide adds a surprising remark: the noise in SGD *actually helps*. A noisy path can shake itself out of small dips (local minima) and often ends up with weights that work better on new data (better generalisation).

The gradient for one example

We need $\nabla \ell$: how the loss of one example changes when each weight changes. Your slide 26 gives the answer, equation (10), and calls it "remarkably simple":

$$\nabla_{\mathbf{w}} \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Read it as two factors. The **error** $(\hat{y}^{(i)} - y^{(i)})$: how far the probability was from the truth, positive if too high, negative if too low. The **direction** $\mathbf{x}^{(i)}$: the input itself. Each weight w_j moves in proportion to its own input $x_j^{(i)}$, with $x_0^{(i)} = 1$ for the bias. Your slide's punchline: this is the *same structure as the linear regression gradient* from Book 3. The sigmoid and the cross-entropy cancel each other's messiness exactly.

Σ MATHS, STEP BY STEP

PLAIN MATHS

Where does $(\hat{y} - y) \mathbf{x}$ come from? Drop the (i) to keep it clean. The loss is $\ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, with $\hat{y} = \sigma(z)$ and $z = \mathbf{w}^\top \mathbf{x}$.

Slope of ℓ with respect to $\hat{y}$:

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}$$

Slope of $\hat{y}$ with respect to z (Chapter 2):

$$\sigma'(z) = \hat{y}(1 - \hat{y}).$$

Multiply (chain rule) and simplify:

$$\frac{\partial \ell}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \right) \hat{y}(1 - \hat{y}) = -y(1 - \hat{y}) + (1 - y)\hat{y} = \hat{y} - y$$

Finally $\partial z / \partial w_j = x_j$, so

$$\frac{\partial \ell}{\partial w_j} = (\hat{y} - y) x_j$$

MATRIX FORM

All $d + 1$ partial derivatives at once, as one vector of shape $(d + 1) \times 1$:

$$\nabla_{\mathbf{w}} \ell = (\hat{y} - y) \mathbf{x}$$

The scalar error multiplies the whole input vector.

Update, one example:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta (\hat{y} - y) \mathbf{x}$$

Batch GD over all N rows is $\nabla J = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y})$: $(d + 1) \times N$ times $N \times 1$ gives $(d + 1) \times 1$. Same formula as Book 3, with σ hidden inside $\hat{\mathbf{y}}$. SGD is this with $N = 1$.

The SGD algorithm

Your slide 27 writes it as Algorithm 1. In words: pick one, forward, error, gradient, update, next one.

1. Start with $\mathbf{w}^{(0)} = \mathbf{0}$ (or small random values).
2. For each epoch $1, \dots, T$: **shuffle** the dataset (so the weights do not pick up a rhythm from a fixed order). Then for each example in turn: compute $\hat{y}^{(i)} = \sigma(\mathbf{w}^\top \mathbf{x}^{(i)})$; compute $\nabla \ell = (\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}$; update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell$.
3. Optionally, at the end of each epoch, compute the total loss $J(\mathbf{w})$ to watch progress.
4. Return $\mathbf{w}$.

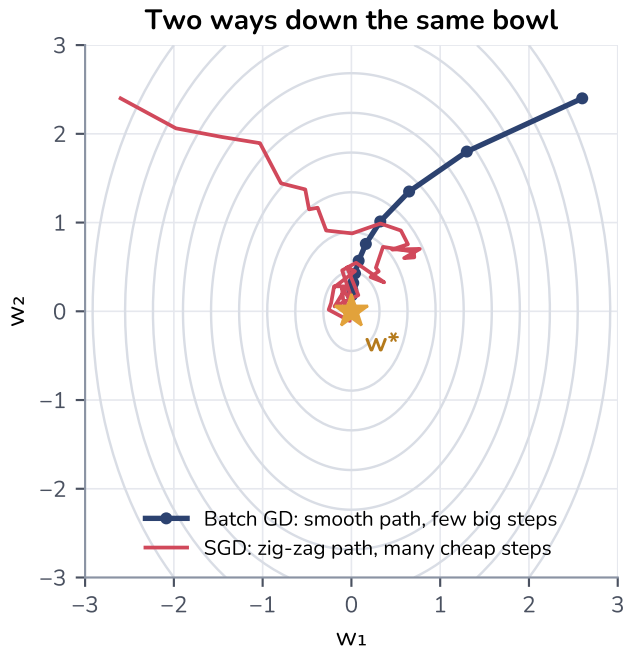
? STOP AND THINK

Ravi: "If SGD's gradient is wrong on every step, how can it ever reach the minimum?"

✓ OUR ANSWER

Each single-example gradient is wrong, but not wrong in a fixed direction. Sometimes it points too far left, sometimes too far right. On *average* over many examples it points the right way, because the average of the per-example gradients is exactly the batch gradient. Many small noisy steps add up to a correct journey, like a drunk person who still walks home because home is downhill.

SGD versus batch GD: the picture



Benefits of SGD

Speed: much faster per iteration (one example). **Memory:** handles data that does not fit in memory. **Online learning:** can update as new data arrives. **Generalisation:** noise helps escape sharp minima. **Scalability:** works with massive datasets.

Challenges of SGD

Noisy updates: the path is not smooth. **Learning rate:** more sensitive to the choice of η . **Convergence:** may oscillate near the minimum instead of settling. **Tuning:** needs careful hyperparameter tuning.

Figure 3.2 Your slide 28. The rings are the loss surface seen from above; the star $\mathbf{w}^*$ is the minimum. Batch GD (blue) takes a smooth, deterministic path: every step uses the exact gradient. SGD (red) zig-zags: every step uses one example, so its direction is a bit wrong each time. But each red step is N times cheaper. Notice that both reach the centre.

Your slide ends with the common compromise: **mini-batch SGD**, a small batch of examples per update. That is Chapter 7.

📄 ON YOUR SLIDES

Slide 21: equation (7) and its two cases. Slide 22: $J(\mathbf{w})$, equation (8), and $\mathbf{w}^* = \arg \min J$, (9). Slide 23: four advantages and the $-\log$ plot. Slide 24: batch GD recap, three problems, SGD. Slide 25: the three-step rule and the noise remark. Slide 26: equation (10), "error", "direction", "same structure as linear regression". Slide 27: Algorithm 1. Slide 28: the two paths. Slide 29: benefits, challenges, "common compromise: mini-batch SGD".

⚠️ CAREFUL

Never feed $\hat{y} = 0$ or $\hat{y} = 1$ into a logarithm: $\log 0$ is minus infinity and your loss becomes NaN. The sigmoid never reaches exactly 0 or 1, but a computer's rounding can. Chapter 10 shows the standard fix (a stable formula for $\log \sigma(z)$).

✅ CHECK YOURSELF

1. The true label is 1 and the model says 0.25. What is the loss? $-\log 0.25 = 1.386$.
2. Why not squared error? **It punishes confident wrong answers far too little, and it is not the natural (maximum-likelihood) loss for probabilities.**
3. What is the SGD gradient for one example? $(\hat{y} - y) \mathbf{x}$: **error times input.**
4. Name two benefits and one challenge of SGD. **Fast, low memory, online; noisy path, sensitive to η .**

🗨️ IN ONE BREATH

- Per example: $\ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, which is $-\log$ of the probability given to the true class.
- Total: $J(\mathbf{w})$ is the average; learning finds $\arg \min J$. Cross-entropy is convex and punishes confident mistakes.
- SGD: pick one random example, step $\mathbf{w} \leftarrow \mathbf{w} - \eta(\hat{y} - y)\mathbf{x}$, repeat. Shuffle every epoch.
- Batch GD: smooth and exact but slow. SGD: noisy and cheap. Mini-batch: the compromise (Chapter 7).

Will Ravi pass? The SGD example by hand

Four students, one feature, two SGD steps done digit by digit, then a check that the curve is moving the right way.

On your slides: **Module 4, slides 31–36** · Numerical example: setup · Initialisation · First SGD iteration · Second SGD iteration · Checking progress · Computational graph for one SGD step

STORY

Ravi has exams next week and has not been studying. "Forget the robot. Will I pass?" Asha grabs the whiteboard. "Give me data." Ravi remembers four friends from last year. One studied 1 hour and failed. One studied 2 hours and failed. One studied 3 hours and passed. One studied 4 hours and passed. "Four students. That is enough for a tiny neuron," says Asha. "Let us train it by hand, one student at a time."

BIG IDEA

Every SGD step is four small moves: **forward** (compute z and $\hat{y}$), **error** ($\hat{y} - y$), **gradient** (error times input), **update** ($\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell$). Do it for one example, then the next. After only two steps the predictions already lean the right way.

THINK OF IT LIKE...

Tuning a guitar string by ear, one pluck at a time. Pluck (forward pass), hear how far it is from the note (error), decide which way to turn the peg and by how much (gradient times learning rate), turn it (update). You do not wait to pluck all six strings before touching a peg.

Setup

The tiny dataset from your slide 31: classify whether a student passes based on study hours.

Hours x_1	Pass y
1	0
2	0
3	1
4	1

One feature, so $d = 1$, and $N = 4$ examples. Add the column of ones for the bias, and stack:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

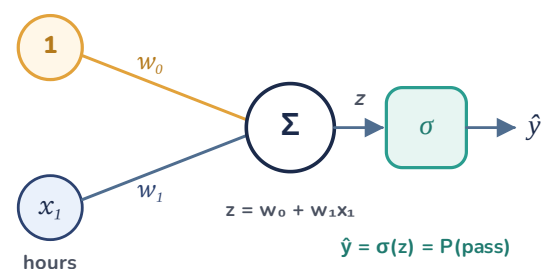


Figure 4.1 The neuron for this problem: one input x_1 (hours), one bias, a sum, a sigmoid. Two weights to learn, w_0 and w_1 .

INITIALISATION (SLIDE 32)

Hyperparameters: learning rate $\eta = 0.5$, and $N = 4$. Start with both weights at zero, $\mathbf{w}^{(0)} = (0, 0)^\top$. With zero weights every logit is $z = 0$, so every prediction is $\sigma(0) = 0.5$:

$$\hat{\mathbf{y}}^{(0)} = (0.5, 0.5, 0.5, 0.5)^\top$$

The model has no opinion yet. It says "50-50" for everyone. The loss function is the cross-entropy from Chapter 3, $\ell(\mathbf{w}; \mathbf{x}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.

WORKED EXAMPLE: FIRST SGD ITERATION (SLIDE 33)

Randomly select the first example: $\mathbf{x}^{(1)} = (1, 1)^\top$, $y^{(1)} = 0$. The student who studied 1 hour and failed.

- 1 Forward pass:** $z^{(1)} = \mathbf{w}^{(0)\top} \mathbf{x}^{(1)} = 0 \times 1 + 0 \times 1 = 0$.
bias times 1, plus weight times hours
- 2** $\hat{y}^{(1)} = \sigma(0) = 0.5$.
the sigmoid of zero is one half
- 3 Error:** $\hat{y}^{(1)} - y^{(1)} = 0.5 - 0 = 0.5$.
predicted minus true; positive means "too high"
- 4 Gradient:** $\nabla \ell = (\hat{y}^{(1)} - y^{(1)}) \mathbf{x}^{(1)} = 0.5 \times (1, 1)^\top = (0.5, 0.5)^\top$.
error times the whole input vector
- 5 Update:** $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla \ell = (0, 0)^\top - 0.5 \times (0.5, 0.5)^\top$.
step against the gradient
- 6** $\mathbf{w}^{(1)} = (0 - 0.25, 0 - 0.25)^\top = (-0.25, -0.25)^\top$.
both weights go down: the model was too high for a "fail" student

WORKED EXAMPLE: SECOND SGD ITERATION (SLIDE 34)

Randomly select the third example: $\mathbf{x}^{(3)} = (1, 3)^\top$, $y^{(3)} = 1$. The student who studied 3 hours and passed.

- 1 Forward pass:** $z^{(3)} = \mathbf{w}^{(1)\top} \mathbf{x}^{(3)} = (-0.25) \times 1 + (-0.25) \times 3 = -0.25 - 0.75 = -1.0$.
- 2** $\hat{y}^{(3)} = \sigma(-1.0) = \frac{1}{1 + e^{1.0}} = \frac{1}{1 + 2.718} = \frac{1}{3.718} \approx 0.269$.
only 27% chance of passing, but this student passed
- 3 Error:** $0.269 - 1 = -0.731$.
negative means "too low"
- 4 Gradient:** $\nabla \ell = -0.731 \times (1, 3)^\top = (-0.731, -2.193)^\top$.
 $-0.731 \times 3 = -2.193$
- 5 Update:** $\mathbf{w}^{(2)} = (-0.25, -0.25)^\top - 0.5 \times (-0.731, -2.193)^\top$.
- 6** $w_0 = -0.25 + 0.3655 = 0.116$; $w_1 = -0.25 + 1.0965 = 0.847$. So $\mathbf{w}^{(2)} = (0.116, 0.847)^\top$.
subtracting a negative gradient pushes both weights up, w_1 most because its input was 3

Checking progress (slide 35)

With $\mathbf{w}^{(2)} = (0.116, 0.847)^\top$, run all four students through the neuron. One line each:

- 1 Example 1:** $\hat{y}^{(1)} = \sigma(0.116 + 0.847 \times 1) = \sigma(0.963) \approx 0.724$.

- 2 Example 2: $\hat{y}^{(2)} = \sigma(0.116 + 0.847 \times 2) = \sigma(1.810) \approx 0.859$.
- 3 Example 3: $\hat{y}^{(3)} = \sigma(0.116 + 0.847 \times 3) = \sigma(2.657) \approx 0.934$.
- 4 Example 4: $\hat{y}^{(4)} = \sigma(0.116 + 0.847 \times 4) = \sigma(3.504) \approx 0.971$.

Your slide reads the progress like this. Examples 3 and 4 (true label 1) are moving towards 1: good. Examples 1 and 2 (true label 0) are, well, 0.724 and 0.859. They are not near 0 yet. But notice the *order* is already right: more hours, higher probability. The curve has the right tilt ($w_1 > 0$); it just needs to slide to the right. Your slide says: continue SGD for multiple epochs until convergence.

! CAREFUL

Two SGD steps are not "training". The total loss after step 2 is actually *higher* than at the start ($J = 0.836$ against 0.693), because step 2 pushed hard on a single "pass" example. That is normal for SGD. Judge progress over epochs, not single steps.

What happens if we keep going

Your slides stop after two steps. We continued in code (see `verify.py`), finishing epoch 1 with examples 2 and 4, then running more epochs in a fixed order 1, 2, 3, 4 so you can follow along. Here is the end of each epoch:

Epoch	w_0	w_1	$\hat{y}^{(1)}$	$\hat{y}^{(2)}$	$\hat{y}^{(3)}$	$\hat{y}^{(4)}$	$J(\mathbf{w})$
0	0.000	0.000	0.500	0.500	0.500	0.500	0.693
after step 2	0.116	0.847	0.724	0.859	0.934	0.971	0.836
1	-0.019	1.168	0.759	0.910	0.970	0.991	0.969
2	-0.433	1.079	0.656	0.849	0.943	0.980	0.759
3	-0.735	1.178	0.609	0.835	0.943	0.982	0.704
5	-1.296	1.352	0.514	0.803	0.940	0.984	0.606
50	-6.560	2.980	0.027	0.354	0.915	0.995	0.140

Watch w_0 . It keeps falling, dragging the whole curve to the right, while w_1 grows and makes it steeper. After 50 epochs the two "fail" students are at 0.027 and 0.354, the two "pass" students at 0.915 and 0.995. All four are on the right side of 0.5. The boundary sits at $z = 0$, that is $-6.56 + 2.98 x_1 = 0$, so $x_1 = 2.2$ hours.

? STOP AND THINK

Ravi: "So do I pass or not? I have studied for 2 hours."

✓ OUR ANSWER

After 50 epochs the model says $\hat{y} = 0.354$ for 2 hours: below 0.5, so it predicts "fail". Study another hour: at 3 hours it says 0.915. Four students is far too little data to trust, but the direction of the advice is sound.

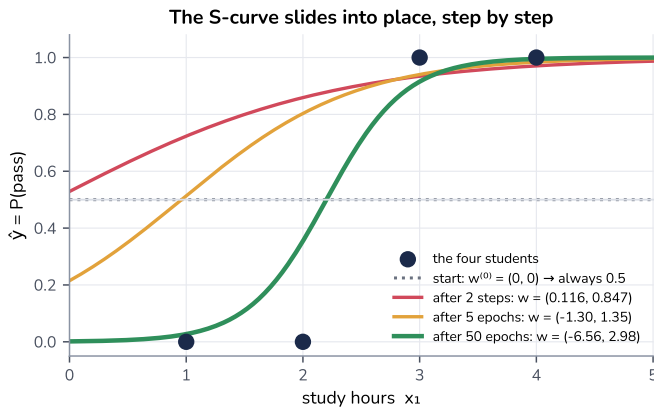


Figure 4.2 The S-curve at four moments. Grey: the flat start. Red: after the two slide steps, tilted the right way but too far left. Green: after 50 epochs, passing cleanly between the failing pair and the passing pair.

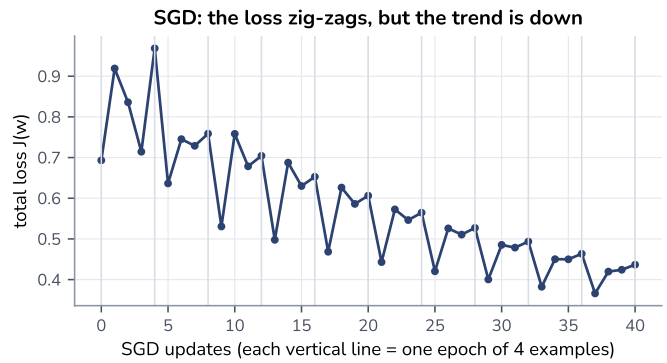


Figure 4.3 The total loss after each single update for ten epochs. It jumps up and down within an epoch (each example pulls its own way) but the trend is clearly down. This is the zig-zag of SGD from Figure 3.2, seen as a graph.

The computational graph for one SGD step

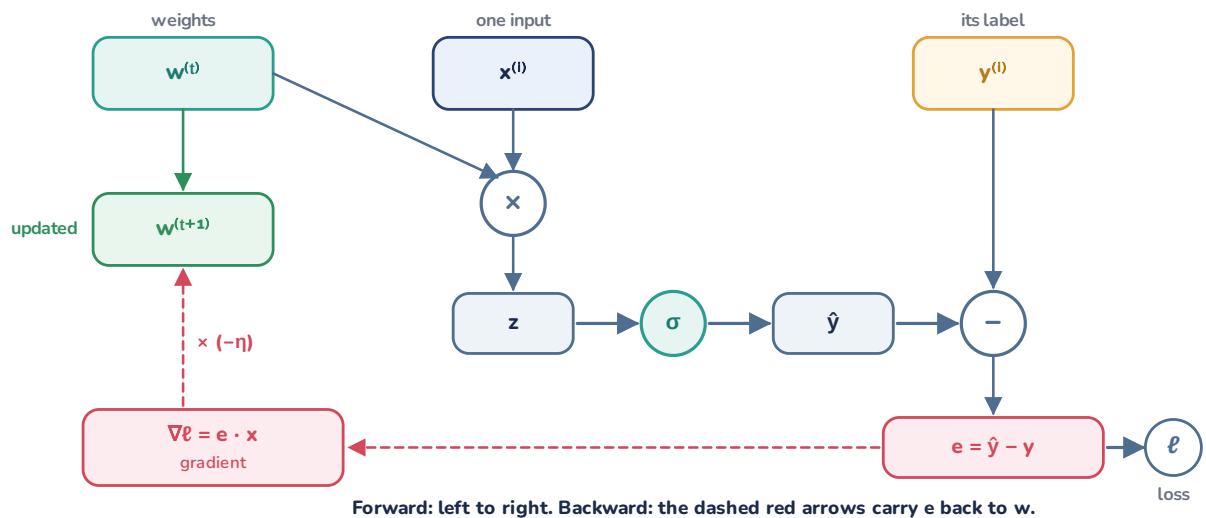


Figure 4.4 Your slide 36 as a flow of boxes. Forward, left to right: the weights and one input multiply to give z . σ turns it into $\hat{y}$. Subtracting the label gives the error e . The loss ℓ is computed from e . Backward, dashed red: the error is multiplied by the input to give the gradient $\nabla \ell = e \cdot \mathbf{x}$, scaled by $-\eta$, and added to the weights. One example per iteration, which is why it is so much faster than batch GD.

Trace the second iteration through this picture. $\mathbf{w}^{(1)} = (-0.25, -0.25)$ and $\mathbf{x}^{(3)} = (1, 3)$ go into $\times$ and give $z = -1.0$. σ gives 0.269. Minus $y = 1$ gives $e = -0.731$. Then $e \cdot \mathbf{x} = (-0.731, -2.193)$. Times -0.5 , added to $\mathbf{w}^{(1)}$, gives $\mathbf{w}^{(2)} = (0.116, 0.847)$. The picture and the numbers are the same story.

ON YOUR SLIDES

Slide 31 has the table, the one-input neuron, and $\mathbf{X}, \mathbf{y}, \mathbf{w}$. Slide 32: $\eta = 0.5$, $N = 4$, $\mathbf{w}^{(0)} = \mathbf{0}$, $\hat{\mathbf{y}}^{(0)} = (0.5, 0.5, 0.5, 0.5)^\top$. Slide 33: first iteration on example 1 in four steps, ending at $\mathbf{w}^{(1)} = (-0.25, -0.25)^\top$. Slide 34: second iteration on example 3, $z = -1.0$, $\hat{y} \approx 0.269$, error -0.731 , gradient $(-0.731, -2.193)^\top$, $\mathbf{w}^{(2)} = (0.116, 0.847)^\top$. Slide 35: the four predictions 0.724, 0.859, 0.934, 0.971 and the progress notes. Slide 36: the computational graph. The rows for epochs 1–50 in the table above are ours, not the deck's.

🔗 TRY IT YOURSELF

Do the third SGD step yourself, on example 2: $\mathbf{x}^{(2)} = (1, 2)^\top$, $y^{(2)} = 0$, starting from $\mathbf{w}^{(2)} = (0.116, 0.847)^\top$. Compute z , $\hat{y}$, the error, the gradient, and $\mathbf{w}^{(3)}$. ($z = 0.116 + 1.694 = 1.810$; $\hat{y} = 0.859$; error 0.859 ; gradient $(0.859, 1.719)$; $\mathbf{w}^{(3)} = (0.116 - 0.430, 0.847 - 0.859) = (-0.314, -0.012)$, or -0.013 without rounding.)

✅ CHECK YOURSELF

1. Why is every initial prediction 0.5? **Zero weights give $z = 0$, and $\sigma(0) = 0.5$.**
2. In iteration 1 the error was $+0.5$. Which way did the weights move? **Down, because we subtract η times a positive gradient.**
3. Why did w_1 change more than w_0 in iteration 2? **Its input was 3, the bias input was 1; the gradient is error times input.**
4. After two steps, which is already correct: the values or the order of the four predictions? **The order (more hours, higher probability). The values still need more epochs.**

🔗 IN ONE BREATH

- Data: hours 1, 2, 3, 4 $\rightarrow$ pass 0, 0, 1, 1. $\eta = 0.5$, $\mathbf{w}^{(0)} = \mathbf{0}$, all predictions start at 0.5.
- Step 1 (example 1): error 0.5, gradient $(0.5, 0.5)$, $\mathbf{w}^{(1)} = (-0.25, -0.25)$.
- Step 2 (example 3): $z = -1$, $\hat{y} = 0.269$, error -0.731 , gradient $(-0.731, -2.193)$, $\mathbf{w}^{(2)} = (0.116, 0.847)$.
- Check: 0.724, 0.859, 0.934, 0.971. Right order, not yet right values. More epochs fix it.

How good is the yes/no machine?

Split the data, count the four kinds of right and wrong, and turn the counts into accuracy, precision, recall and F1.

On your slides: **Module 4, slides 38–45** · Evaluating classification models · Confusion matrix · Metrics I and II · When to use which metric · Computing metrics · Summary · Key takeaways

STORY

Nova has trained on thousands of clips. Asha wants one number: how good is it? She plays eight clips it has never heard: four with her name, four without. Nova gets six right. "75%," says Ravi. "But which two did it miss?" Asha looks. One time Nova stayed asleep when she called it. One time it woke up for the neighbour's dog. "Those two mistakes feel very different," she says.

BIG IDEA

A yes/no guess can go wrong in two ways. Saying yes when the truth is no is a **false positive**. Saying no when the truth is yes is a **false negative**. The **confusion matrix** counts all four outcomes. Accuracy, precision, recall and F1 are four ways of summarising those counts. Which one matters depends on which mistake costs more.

THINK OF IT LIKE...

A security guard at a hostel gate checking for outsiders. *Precision*: of all the people he stopped, how many were really outsiders? *Recall*: of all the outsiders who came, how many did he stop? A lazy guard who stops nobody never bothers a resident, but has zero recall. A paranoid guard who stops everybody has perfect recall but terrible precision. You want both, and F1 rewards both.

Three questions, and a data split

Your slide 38 starts with the key questions. How often does the model predict correctly? That is **accuracy**. What kinds of errors does it make? That is the **confusion matrix**. How well does it find the positive cases? That is **precision** and **recall**.

Before any of that, split the data, as in Book 3:

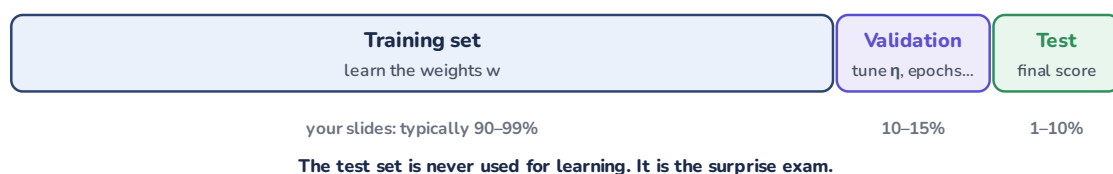


Figure 5.1 The three sets and what each is for. Your slide gives typical sizes: training 90–99%, validation 10–15%, test 1–10% (the point is that training gets the lion's share). Always evaluate on unseen test data: that is the only way to measure generalisation, how well the model does on new examples.

The confusion matrix

		Predicted	
		Positive (1)	Negative (0)
Actual	Positive (1)	True Positive TP right	False Negative FN missed it (Type II)
	Negative (0)	False Positive FP false alarm (Type I)	True Negative TN right

Green = the guess was right. Red = the guess was wrong.

Figure 5.2 Rows are the truth, columns are the model's guess. The two green cells are right answers; the two red cells are the two kinds of mistake. Notice the names: the first word says whether the guess was right (True/False), the second word says what the guess was (Positive/Negative).

- **TP, true positive:** truth 1, guessed 1. Correctly predicted positive.
- **TN, true negative:** truth 0, guessed 0. Correctly predicted negative.
- **FP, false positive:** truth 0, guessed 1. A false alarm. Your slide calls this a **Type I error**.
- **FN, false negative:** truth 1, guessed 0. A miss. A **Type II error**.

Nova waking for the dog is a false positive; Nova sleeping through Asha's call is a false negative.

Four metrics

Σ MATHS, STEP BY STEP

THE FORMULAS (EQUATIONS 11–14)

Accuracy, overall correctness: right answers over everything.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: of the predicted positives, how many were correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (also called sensitivity): of the actual positives, how many did we find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 score: the harmonic mean of precision and recall.

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

HOW TO READ THEM

Precision and recall share the top, *TP*. They differ in the bottom. Precision divides by the *predicted* positives (a column of the matrix). Recall divides by the *actual* positives (a row).

The **harmonic mean** is a mean that is dragged down by the smaller number. If precision is 1.0 and recall is 0.1, the ordinary average is 0.55, but F1 is $2 \times \frac{0.1}{1.1} = 0.18$. F1 refuses to reward a model that is good at one and hopeless at the other.

All four lie between 0 and 1; higher is better.

WHEN TO USE WHICH (SLIDE 42)

Accuracy

Use when the classes are balanced. Misleading with imbalanced data: if 95% of emails are not spam, a model that says "not spam" to everything scores 95% accuracy and catches nothing.

Precision

Important when false positives are costly. Spam detection: you do not want a real email marked as spam and lost.

Recall

Important when false negatives are costly. Disease detection: you do not want to miss a sick patient.

F1 score

Use when you need a balance between precision and recall. Good for imbalanced datasets.

WORKED EXAMPLE: COMPUTING THE METRICS (SLIDE 43)

Eight test examples. Four have true label 1, four have true label 0. The model's guesses, with each one tagged:

clip 1 true 1 guess 1 TP	clip 2 true 1 guess 1 TP	clip 3 true 1 guess 0 FN	clip 4 true 1 guess 1 TP	clip 5 true 0 guess 0 TN	clip 6 true 0 guess 0 TN	clip 7 true 0 guess 1 FP	clip 8 true 0 guess 0 TN
--	--	--	--	--	--	--	--

Counts: $TP = 3$, $TN = 3$, $FP = 1$, $FN = 1$ ($N = 8$)

- 1 Count: $TP = 3$, $TN = 3$, $FP = 1$, $FN = 1$.
three green cards of each kind, one red card of each kind
- 2 Accuracy = $\frac{3 + 3}{3 + 3 + 1 + 1} = \frac{6}{8} = 0.75$.
- 3 Precision = $\frac{3}{3 + 1} = 0.75$.
of the 4 times it said "1", 3 were right
- 4 Recall = $\frac{3}{3 + 1} = 0.75$.
of the 4 real "1"s, it found 3
- 5 $F_1 = 2 \cdot \frac{0.75 \times 0.75}{0.75 + 0.75} = 2 \cdot \frac{0.5625}{1.5} = 0.75$.
when precision equals recall, F1 equals them too

All four metrics agree here because $FP = FN$. Change one count and they split apart (see Check yourself).

KEY TAKEAWAYS (SLIDE 45)

- Binary classification predicts discrete categories, $y \in \{0, 1\}$. The sigmoid $\sigma(z) = 1/(1 + e^{-z})$ maps any number to a probability in $[0, 1]$.
- Logistic regression = linear model + sigmoid activation. Cross-entropy is the right loss for classification.
- SGD updates the weights one example at a time: faster and more scalable. Feature scaling is essential for convergence and numerical stability (Chapter 10).

- Evaluation: accuracy, precision, recall, F1, confusion matrix. The same framework extends to multi-class classification (next chapter) and to deep networks (Book 5).

Summary of binary classification

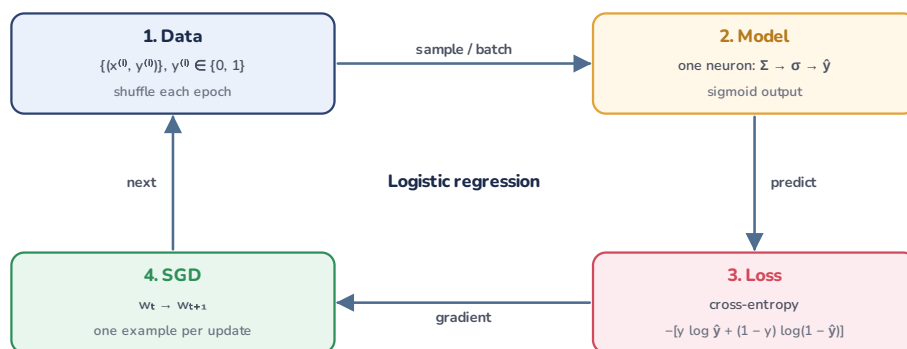


Figure 5.3 Your slide 44: the four components as one loop. Data (shuffled each epoch) feeds one example to the model (Σ , then σ). The model predicts. Cross-entropy scores the prediction. SGD uses the gradient to move $\mathbf{w}_t$ to $\mathbf{w}_{t+1}$. Then back to the data for the next example. Your slide's caption: this framework is the foundation for neural networks.

ON YOUR SLIDES

Slide 38: the three questions and the split (training 90–99%, validation 10–15%, test 1–10%). Slide 39: the confusion matrix with TP, FN, FP, TN and Type I / Type II. Slides 40–41: equations (11)–(14). Slide 42: when to use which. Slide 43: the eight predictions, $TP = 3, TN = 3, FP = 1, FN = 1$, all four metrics 0.75. Slide 44: the summary loop. Slide 45: key takeaways.

CAREFUL

Rows and columns get swapped in different books. Your deck puts *actual* on the rows and *predicted* on the columns. Read the labels before you count.

CHECK YOURSELF

1. Nova wakes up for a door slam. TP, TN, FP or FN? **FP: guessed positive, truth negative.**
2. A cancer test must not miss sick patients. Which metric? **Recall.**
3. Precision is TP divided by what? **$TP + FP$, all predicted positives.**
4. Why is 95% accuracy possibly worthless? **If 95% of examples are negative, always saying "negative" gets 95%.**
5. With $TP = 3, TN = 4, FP = 0, FN = 1$, what are precision, recall and F1? **1.0, 0.75, $2 \times 0.75 / 1.75 = 0.857$.**

IN ONE BREATH

- Split into training, validation, test. Score only on the test set.
- Confusion matrix: TP, TN (right), FP (false alarm, Type I), FN (miss, Type II).
- Accuracy = $\frac{TP+TN}{\text{all}}$, precision = $\frac{TP}{TP+FP}$, recall = $\frac{TP}{TP+FN}$, F_1 = harmonic mean of the last two.
- Costly false alarms → precision. Costly misses → recall. Imbalance → F1, never accuracy alone.

Many choices: the softmax neuron

One neuron per class, then softmax to turn K scores into K probabilities that add up to one.

On your slides: **Module 4, slides 47–63** · Multi-class classification · Examples · Visual · Five components · Data representation · One-hot encoding · Weight matrix · Architecture · Logits · Softmax · Properties · Numerical example · Forward pass · Categorical cross-entropy

STORY

Yes/no is not enough. Asha wants Nova to tell three sounds apart: her name, a clap, and silence. "Three neurons," says Ravi. "One for each." Asha tries it. For one clip the three neurons say 2.0, 1.0 and 0.1. "Which one wins?" "The first. But how sure is it? These are not probabilities. They do not even add up to 1." Asha needs a way to turn three raw scores into three shares of a whole.

BIG IDEA

For K classes use K linear neurons, one per class, each producing a raw score called a **logit**. Then apply **softmax**: take e to the power of each logit and divide by the sum. The results are K positive numbers that add up to 1, so they are probabilities. The loss, **categorical cross-entropy**, is $-\log$ of the probability given to the true class, exactly as in the binary case.

THINK OF IT LIKE...

Three friends vote on where to eat, but each shouts with a different loudness: 2.0, 1.0, 0.1. To turn shouting into shares, you make every voice positive and exaggerate the loud ones (that is e^z), then divide each by the total noise. Now the shares are 66%, 24% and 10%, and they add up to 100%. The loudest voice gets the biggest share, but the others are not silenced. That is softmax: "soft" because it does not simply pick the maximum.

What multi-class classification is

Your slide 47: multi-class classification assigns each input to one of K classes, where $K > 2$. The classes are $y \in \{1, 2, \dots, K\}$ (some books count from 0). Each example belongs to *exactly one* class. The output is a **probability distribution** over all K classes, so the probabilities sum to 1: $\sum_{k=1}^K P(y = k | \mathbf{x}) = 1$. For a handwritten digit, the output is ten probabilities, one per digit, and the prediction is the class with the highest one.

Problem (slide 48)	Input	Output
Handwritten digits (MNIST)	a 28×28 pixel image	one of 10 digits, 0–9
Image classification (ImageNet)	a colour image	one of 1000 object categories
Text classification	a news article	a topic: politics, sports, technology, entertainment, business
Speech recognition	audio features	a phoneme class (the small sound units of a language)

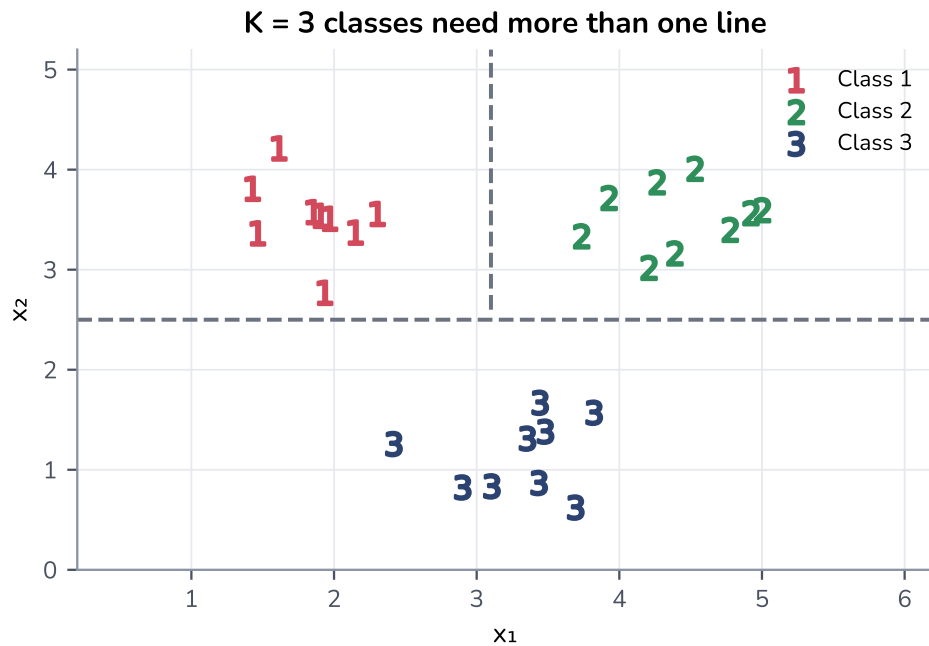


Figure 6.1 Your slide 49: three groups in feature space, and the boundaries that separate $K = 3$ classes. One line is no longer enough. The key insight on your slide: we need *multiple output neurons*, one per class, and a way to turn their outputs into a valid probability distribution.

THE FIVE COMPONENTS (SLIDE 51)

Your deck lists five parts this time, because it splits the model from its activation.

1. Data features and labels $y \in \{1, \dots, K\}$	2. Model linear, K output neurons, no hidden layer	3. Activation softmax, gives a distribution	4. Objective categorical cross-entropy	5. Learning mini-batch SGD (Chapter 7)
---	--	---	--	--

Data: integers or one-hot

The inputs are the same design matrix as before, $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$, a row per example with a leading 1. The labels can be written two ways.

- Integer encoding:** one number per example, $\mathbf{y} = (3, 1, 2, 1, 3)^\top$ on your slide, $\mathbf{y} \in \{1, \dots, K\}^N$.
- One-hot encoding:** a matrix $\mathbf{Y} \in \{0, 1\}^{N \times K}$. Each row is all zeros except a single 1 in the column of the true class. Your deck says this is preferred for computation.

integer label		one-hot vector ($K = 4$)					
class 1	→	<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	$\mathbf{y} = [1, 0, 0, 0]^\top$
1	0	0	0				
class 2	→	<table><tr><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	0	1	0	0	$\mathbf{y} = [0, 1, 0, 0]^\top$
0	1	0	0				
class 3	→	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td></tr></table>	0	0	1	0	$\mathbf{y} = [0, 0, 1, 0]^\top$
0	0	1	0				
class 4	→	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	0	0	1	$\mathbf{y} = [0, 0, 0, 1]^\top$
0	0	0	1				

one row per example, one column per class, a single 1 in the true class's column

Figure 6.2 Your slide 53 with $K = 4$. Class 2 becomes $[0, 1, 0, 0]^\top$: a 1 in position 2, zeros elsewhere. Notice that each one-hot vector looks exactly like a "perfectly confident" softmax output. That is the point.

Why one-hot? Your slide gives four reasons. It treats all classes equally (there is no hidden ordering: class 3 is not "more" than class 1). It matches the format of the softmax output, so prediction and truth can be compared entry by entry. It simplifies the loss computation. And it is standard in deep learning frameworks.

The weight matrix

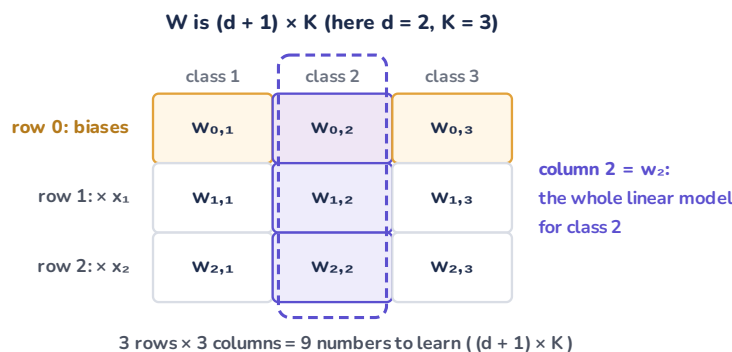


Figure 6.3 $\mathbf{W}$ for $d = 2$ features and $K = 3$ classes. Each *column* $\mathbf{w}_k$ is a complete linear model for class k , with its own bias in row 0. The dashed column is class 2's model. Notice that binary classification had one weight vector; now there are K of them, side by side.

For K classes and d features the weights form a matrix $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$. Entry $w_{j,k}$ is the weight from input j to class k . Row 0 holds the biases for all classes. Total parameters: $(d + 1) \times K$. For MNIST that is $785 \times 10 = 7850$ numbers.

Architecture and logits

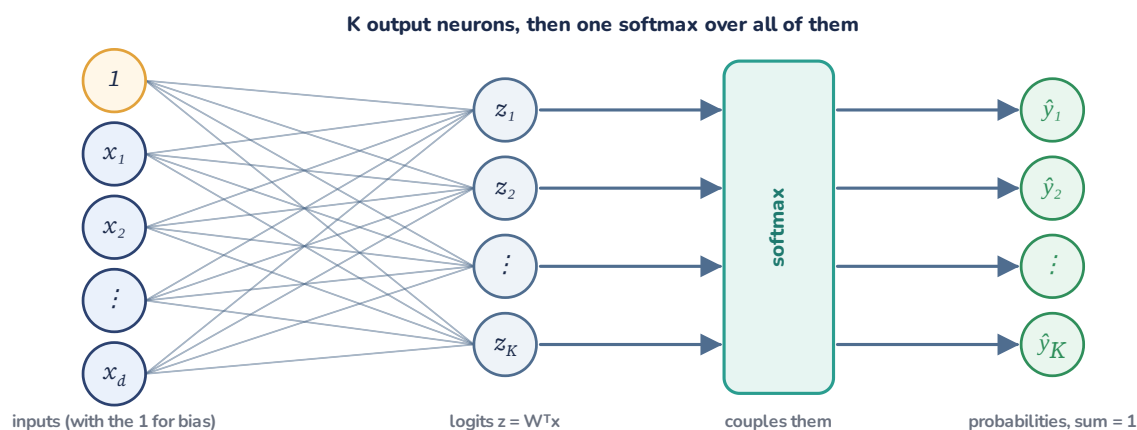


Figure 6.4 Your slide 55. Every input connects to every output neuron. The K neurons compute K logits $z_1, \dots, z_K$ independently. The softmax box then looks at all of them together and produces $\hat{y}_1, \dots, \hat{y}_K$. Notice that softmax is one box across all outputs, not K separate boxes: it *couples* the outputs.

PLAIN MATHS, ONE INPUT

Class k has its own weight column $\mathbf{w}_k$. Its logit is a plain weighted sum:

$$z_k = \mathbf{w}_k^\top \mathbf{x} = w_{0,k} + w_{1,k}x_1 + \cdots + w_{d,k}x_d$$

All K logits at once, equation (15):

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} = \begin{bmatrix} \mathbf{w}_1^\top \mathbf{x} \\ \vdots \\ \mathbf{w}_K^\top \mathbf{x} \end{bmatrix} = \begin{bmatrix} z_1 \\ \vdots \\ z_K \end{bmatrix}$$

$\mathbf{W}^\top$ is $K \times (d+1)$, $\mathbf{x}$ is $(d+1) \times 1$, so $\mathbf{z}$ is $K \times 1$. Logits are unbounded, in $(-\infty, \infty)$: not probabilities yet.

Softmax, equation (17), for each k :

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

and $\hat{y}_k = P(y = k | \mathbf{x})$ is that value, equation (18).

MATRIX FORM, A WHOLE BATCH

Stack N inputs as rows of $\mathbf{X}$. Equation (16):

$$\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{N \times K}$$

$\mathbf{X}$ is $N \times (d+1)$, $\mathbf{W}$ is $(d+1) \times K$: one multiplication gives every logit of every example. Row i of $\mathbf{Z}$ holds the K logits of example i .

The complete forward pass, equations (19)–(21):

$$\mathbf{Z} = \mathbf{XW}, \quad \hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in [0, 1]^{N \times K}$$

$$\hat{k} = \arg \max_k \hat{y}_k$$

Softmax is applied *row-wise*, each example on its own, so each row of $\hat{\mathbf{Y}}$ is a distribution over K classes. $\arg \max$ means "the index of the biggest entry": the predicted class.

SIX PROPERTIES OF SOFTMAX (SLIDE 58)

1. **Output range:** every $\hat{y}_k \in (0, 1)$.
2. **Probability distribution:** $\sum_{k=1}^K \hat{y}_k = 1$.
3. **Preserves order:** if $z_i > z_j$ then $\hat{y}_i > \hat{y}_j$. The biggest logit stays the biggest probability.
4. **Translation invariant:** $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c)$ for any constant c . Adding the same number to every logit changes nothing (Chapter 10 uses this to avoid overflow).
5. **Differentiable:** gradient-based learning works.
6. **Reduces to sigmoid:** when $K = 2$, softmax is the sigmoid in disguise.

Your slide adds: the exponential *amplifies* differences. Larger logits get *much* larger probabilities.

WORKED EXAMPLE: SOFTMAX BY HAND (SLIDE 59)

Logits $\mathbf{z} = (2.0, 1.0, 0.1)^\top$ for $K = 3$ classes, Nova's three scores for one clip.

1. **Exponentials:** $e^{2.0} \approx 7.39$, $e^{1.0} \approx 2.72$, $e^{0.1} \approx 1.11$.
all positive now; the gaps have grown
2. **Sum:** $7.39 + 2.72 + 1.11 = 11.22$.
3. **Normalise:** $\hat{y}_1 = 7.39/11.22 \approx 0.659$, $\hat{y}_2 = 2.72/11.22 \approx 0.242$, $\hat{y}_3 = 1.11/11.22 \approx 0.099$.
each share of the total; they add to 1.000
4. **Predicted class:** $\arg \max_k \hat{y}_k = 1$.
"Nova", with 66% confidence

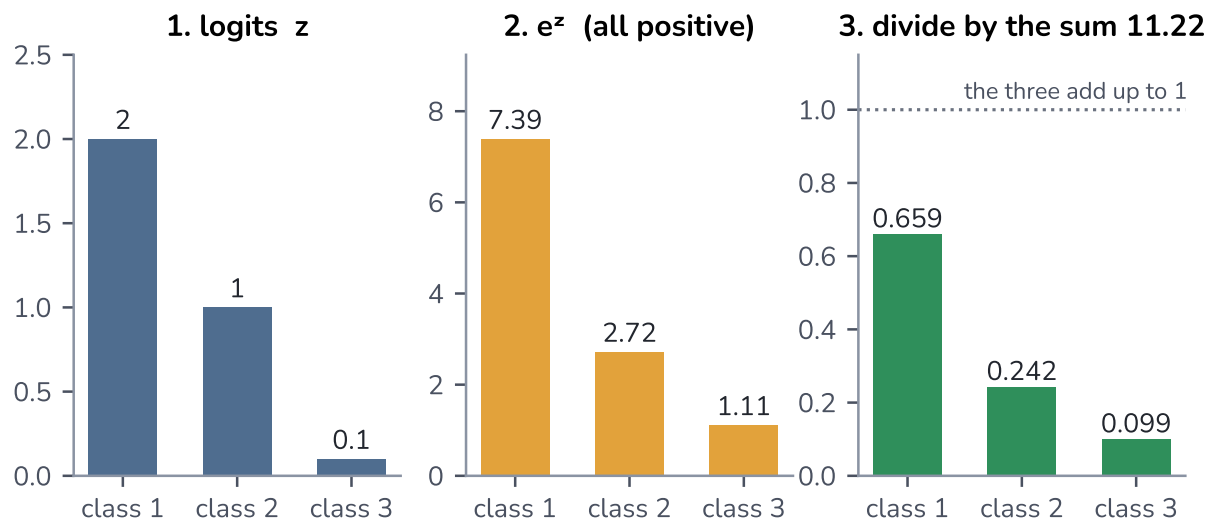


Figure 6.5 The three steps as bars. Left: the logits. Middle: after e^z , the tallest bar has pulled far ahead (7.39 versus 2.72), and every bar is positive. Right: after dividing by 11.22 the bars are shares that add up to 1.

Categorical cross-entropy

With one-hot labels the loss for the whole dataset is equation (22):

$$J(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)})$$

The inner sum looks like K terms, but $y_k^{(i)}$ is 1 for exactly one k and 0 for the rest. So for each example only one term survives: $-\log$ of the probability given to the true class. The loss depends *only* on that one probability; it does not care how the remaining probability is spread among the wrong classes. Goal: find $\mathbf{W}^*$ that minimises $J(\mathbf{W})$.

Your slide 62 shows this with the true class 2, $\mathbf{y} = [0, 1, 0]$:

Prediction	$\hat{y}_1$	$\hat{y}_2$	Loss = $-\log \hat{y}_2$
Good	0.1	0.8	0.22
Medium	0.3	0.5	0.69
Bad	0.7	0.2	1.61
Very bad	0.9	0.05	3.00

The loss grows faster and faster as the true class's probability shrinks: from 0.8 to 0.5 costs 0.47, from 0.2 to 0.05 costs 1.39.



Figure 6.6 The four rows of the table on the $-\log$ curve. Notice how steep it becomes near zero.

WHY CATEGORICAL CROSS-ENTROPY? (SLIDE 63)

The same five reasons as before, now for K classes. It comes from maximum likelihood. It handles K classes naturally. It works well with softmax: the gradient is numerically stable, and as simple as $\hat{\mathbf{y}} - \mathbf{y}$. It heavily penalises confident mistakes. And it is convex for a linear model like ours.

And the connection to Chapter 3. When $K = 2$, the categorical loss is $-[y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2]$. Write $y_2 = y$ and $y_1 = 1 - y$, and $\hat{y}_1 = 1 - \hat{y}_2$: you get $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, the binary cross-entropy. They are the same loss.

STOP AND THINK

Ravi: "Why not just divide each logit by the sum of the logits? Why the e ?"

OUR ANSWER

Logits can be negative or zero, and dividing by a sum of mixed signs gives nonsense (negative "probabilities", or division by zero). e^z is always positive, so the shares are always valid. It is also smooth, which learning needs, and it makes the biggest logit stand out, which is what we want from a confident classifier.

ON YOUR SLIDES

Slide 47: definition, $\sum_k P(y = k | \mathbf{x}) = 1$, the digit example. Slide 48: the four examples. Slide 49: the three-class picture and the key insight. Slide 51: the five components. Slide 52: $\mathbf{X}$, integer labels $\mathbf{y} = (3, 1, 2, 1, 3)^\top$, one-hot $\mathbf{Y}$. Slide 53: one-hot with $K = 4$ and four reasons. Slide 54: $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$. Slide 55: the architecture. Slide 56: equations (15)–(16). Slides 57–58: softmax (17)–(18) and six properties. Slide 59: the (2.0, 1.0, 0.1) example. Slide 60: equations (19)–(21). Slide 61: equation (22). Slide 62: the good/medium/bad/very bad table. Slide 63: five advantages and $K = 2$.

CAREFUL

The logits are not probabilities, even when they happen to lie between 0 and 1. Only after softmax do you have probabilities. And softmax must be applied to all K logits of one example together, never to one logit alone.

CHECK YOURSELF

1. How many parameters does a softmax classifier have for d features and K classes? $(d + 1) \times K$.
2. What is the one-hot vector for class 3 of $K = 4$? $[0, 0, 1, 0]^\top$.
3. Logits (1, 1, 1). What does softmax give? $(1/3, 1/3, 1/3)$.
4. True class 2, prediction (0.3, 0.5, 0.2). What is the loss? $-\log 0.5 = 0.69$.
5. Does the loss change if the prediction becomes (0.1, 0.5, 0.4)? **No. Only $\hat{y}_2$ matters.**

IN ONE BREATH

- K classes, exactly one true class per example. Labels as one-hot rows of $\mathbf{Y} \in \{0, 1\}^{N \times K}$.
- Weights $\mathbf{W}$, $(d + 1) \times K$, one column per class. Logits $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$; for a batch $\mathbf{Z} = \mathbf{X}\mathbf{W}$.
- Softmax: $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$. Positive, sums to 1, keeps order, ignores a common shift. $(2.0, 1.0, 0.1) \rightarrow (0.659, 0.242, 0.099)$.
- Loss: $J = -\frac{1}{N} \sum_i \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$, which is $-\log$ of the true class's probability. $K = 2$ gives back binary cross-entropy.

Learning in small batches

Not one example, not all of them: a handful at a time. Mini-batch SGD is the standard way to train.

On your slides: **Module 4, slides 65–71** · From SGD to mini-batch SGD · Why mini-batch · Key concepts · Computing gradients · Update rule · Algorithm · Choosing batch size

STORY

Asha now has 60,000 labelled sound clips. Batch GD needs all 60,000 before every single step; the laptop fan screams and nothing moves. SGD takes one clip per step. It moves, but the weights jitter about like a nervous bird. And the laptop's graphics chip sits idle, because one clip is too little work for it. Ravi: "Why not a tray of clips at a time? Say 128?" Asha: "That is exactly what everyone does."

BIG IDEA

Mini-batch SGD takes B examples per update, with $1 < B \ll N$. The gradient is the average over the batch: $\frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$. It is one matrix multiplication, so the hardware is busy, the gradient is less noisy than one example's, and the memory holds only B rows. One pass over all $\lceil N/B \rceil$ batches is an **epoch**.

THINK OF IT LIKE...

Marking exam scripts. Marking all 60,000 before writing a single feedback note is batch GD: exact, but the feedback arrives next year. Marking one script and immediately rewriting the course is SGD: fast, but one odd script can send you the wrong way. Marking a stack of 128, noting what the stack had in common, then adjusting: that is a mini-batch. Big enough to see a pattern, small enough to act often.

Three ways to use the data

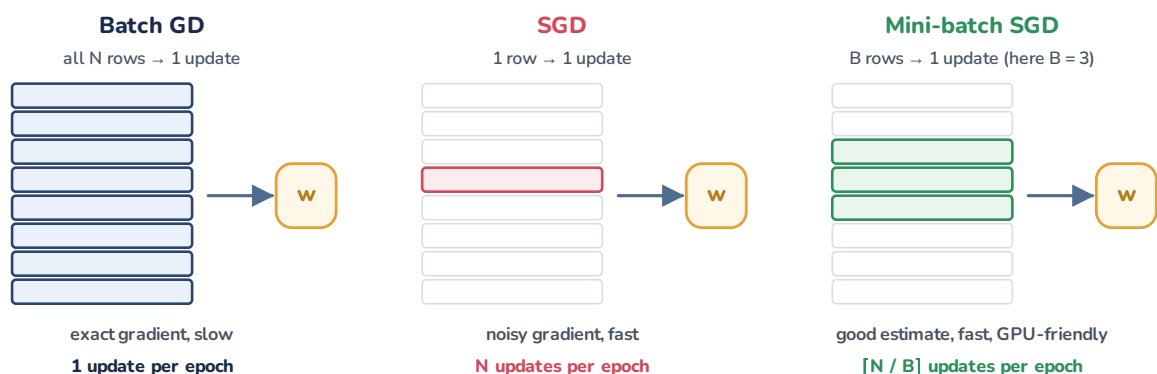


Figure 7.1 Your slide 65. Batch GD: all N rows go into one update. SGD: one row per update. Mini-batch SGD: B rows per update. Notice the trade: more rows per update means a more accurate gradient but fewer updates per epoch.

1. **Batch gradient descent:** all N examples per update. Slow but stable, with an accurate gradient.

2. **Stochastic gradient descent**: one random example per update. Fast but noisy and unstable.
3. **Mini-batch SGD**, the best of both: a small batch of B examples per update, $1 < B \ll N$. Typical sizes $B \in \{32, 64, 128, 256, 512\}$. A good balance of speed and stability, and the standard in modern deep learning.

WHY MINI-BATCHES? (SLIDE 66)

1. Computational efficiency

The whole batch is one matrix multiplication (vectorised). That keeps a GPU busy. Faster than processing examples one by one.

2. Gradient estimation

Averaging B examples is less noisy than a single example. More stable convergence; a better approximation of the true gradient.

3. Generalisation

Some noise remains, and that helps escape sharp minima. Often gives better test performance than exact batch GD.

4. Memory efficiency

Only B rows in memory at once. Datasets larger than memory are loaded and processed batch by batch.

Batch size, iteration, epoch

Three words your deck defines on slide 67. The **batch size** B is the number of examples per update. Small (32–64): more noise, faster iterations. Medium (128–256): balanced. Large (512–1024): less noise but slower convergence. An **iteration** is one weight update, one mini-batch processed. An **epoch** is one complete pass through the whole dataset. Equation (23):

$$\text{Iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

The brackets $\lceil \cdot \rceil$ mean "round up": the last batch may be smaller, but it still counts as an iteration.

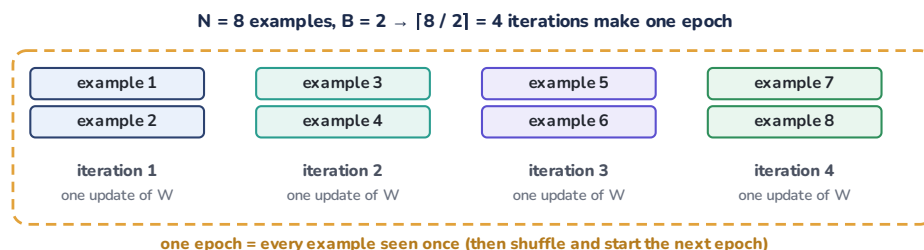


Figure 7.2 A toy dataset of 8 examples with $B = 2$. Each coloured pair is one mini-batch, one iteration, one update. All four together are one epoch. Then shuffle and go again.

WORKED EXAMPLE: HOW MANY UPDATES? (SLIDE 67)

$N = 60,000$ samples (Asha's clips, or MNIST), batch size $B = 128$.

- 1 $60,000 / 128 = 468.75$.
- 2 Round up: $\lceil 468.75 \rceil = 469$ iterations per epoch.
468 full batches of 128, plus one last batch of 96
- 3 After 10 epochs: $469 \times 10 = 4690$ weight updates.
batch GD would have made only 10

The mini-batch gradient and update

Σ MATHS, STEP BY STEP

PLAIN MATHS

Let $\mathcal{B}$ be a mini-batch of B examples. The batch gradient is the *average* of the per-example gradients, equation (24):

$$\nabla J_{\mathcal{B}}(\mathbf{W}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell(\mathbf{W}; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

For softmax with categorical cross-entropy, each example's gradient is (error) times (input), just as in Chapters 3 and 4. But now the error is a whole row, $\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}$: K numbers, one per class. The gradient for weight $w_{j,k}$ is

$$\frac{\partial \ell}{\partial w_{j,k}} = (\hat{y}_k - y_k) x_j$$

error of class k times input j .

MATRIX FORM

Stack the batch. Equation (25):

$$\nabla_{\mathbf{W}} J_{\mathcal{B}} = \frac{1}{B} \mathbf{X}_B^{\top} (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$$

Shapes: $\mathbf{X}_B \in \mathbb{R}^{B \times (d+1)}$ (batch inputs), $\hat{\mathbf{Y}}_B \in \mathbb{R}^{B \times K}$ (predicted probabilities), $\mathbf{Y}_B \in \mathbb{R}^{B \times K}$ (one-hot truth). So $\mathbf{X}_B^{\top}$ is $(d+1) \times B$, times $B \times K$, giving $(d+1) \times K$: the same shape as $\mathbf{W}$, as it must be.

Update, equations (26)–(27):

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla J_{\mathcal{B}}(\mathbf{W}^{(t)}) = \mathbf{W}^{(t)} - \frac{\eta}{B} \mathbf{X}_B^{\top} (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$$

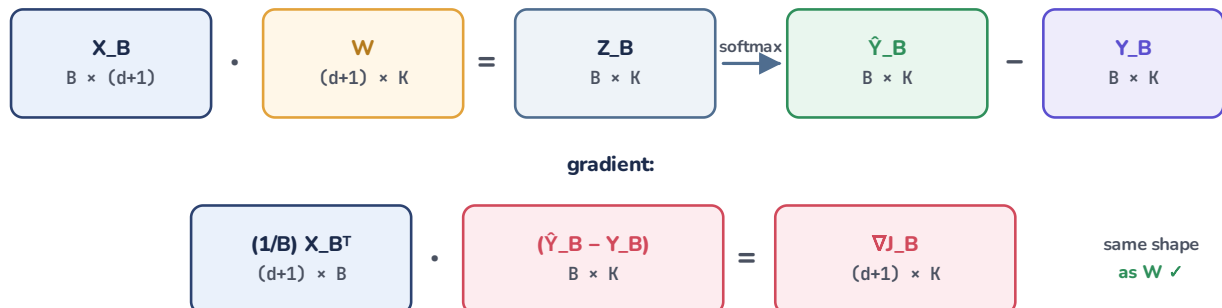


Figure 7.3 Every matrix in one mini-batch step, with its shape. Top row: forward, $\mathbf{X}_B \mathbf{W} = \mathbf{Z}_B$, softmax, then subtract $\mathbf{Y}_B$. Bottom row: the gradient. Notice that the shapes chain together: the inner sizes always match, and the result has the shape of $\mathbf{W}$.

The algorithm (slide 70)

Algorithm 2 on your slide, in words. Input: the dataset, batch size B , learning rate η , number of epochs T . Output: the learned $\mathbf{W}$.

1. Initialise $\mathbf{W} \sim \mathcal{N}(0, 0.01)$: small random numbers drawn from a normal (bell-shaped) distribution centred at 0 with a small spread.
2. For each epoch: **shuffle** the dataset. For each mini-batch of size B :
 - forward: $\mathbf{Z}_B = \mathbf{X}_B \mathbf{W}$ (logits), $\hat{\mathbf{Y}}_B = \text{softmax}(\mathbf{Z}_B)$;
 - gradient: $\nabla J = \frac{1}{B} \mathbf{X}_B^{\top} (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$;
 - update: $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla J$.
3. Optionally compute the total loss to monitor progress. Return $\mathbf{W}$.

Compare with Algorithm 1 in Chapter 3: identical, except that "one example" became "one batch" and every vector became a matrix.

Choosing the batch size (slide 71)

Batch size	Advantages	Disadvantages
Small (32–64)	fast iterations; more updates per epoch; better generalisation; escapes sharp minima	noisy gradients; unstable training; poor GPU use
Medium (128–256)	good balance; stable training; good GPU usage; the standard choice	may need tuning
Large (512–1024)	stable gradients; better GPU use; faster per-epoch time	slow convergence; may generalise worse; more memory

? STOP AND THINK

Ravi: "Bigger batches give a better gradient. So why not always use the biggest batch that fits?"

✓ OUR ANSWER

You pay twice: each update takes longer, and there are fewer updates per epoch. A slightly noisy gradient applied 469 times often beats a perfect one applied 10 times. And large batches may generalise worse: a little noise is useful. (Powers of two suit computer memory.)

☰ ON YOUR SLIDES

Slide 65: three variants. Slide 66: four advantages. Slide 67: batch size, iteration, epoch, (23), 469 and 4690. Slide 68: (24)–(25) with shapes. Slide 69: (26)–(27). Slide 70: Algorithm 2. Slide 71: the batch-size table.

! CAREFUL

Iteration $\neq$ epoch. With $B = 128$, $N = 60,000$: one epoch is 469 iterations, so "100 epochs" means 46,900 updates.

✓ CHECK YOURSELF

1. $N = 1000$, $B = 64$. Iterations per epoch? $\lceil 15.6 \rceil = 16$.
2. What is the shape of $\mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$? $(d+1) \times K$, same as $\mathbf{W}$.
3. Why start $\mathbf{W}$ small and random? **Small avoids saturation; random gives each class its own start** (Chapter 10).

🔴 IN ONE BREATH

- Batch GD: all N . SGD: 1. Mini-batch: B per update (32–512). Vectorised, less noisy, fits in memory.
- Iterations per epoch = $\lceil N/B \rceil$; $60000/128 \rightarrow 469$; 10 epochs $\rightarrow$ 4690 updates.
- Gradient $\frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$, shape $(d+1) \times K$. Update $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla J$. Init $\mathcal{N}(0, 0.01)$, shuffle.

The mini-batch example, by hand

Four points, three classes, one mini-batch of two: every matrix multiplied out, then a trained model used on a new input.

On your slides: **Module 4, slides 73–82** · Numerical example: setup · Data representation · First mini-batch · Softmax computation · Gradient and update · Weight update · Computational graph · Inference · Inference example

STORY

Asha writes four sounds on the whiteboard, each described by two numbers: loudness x_1 and pitch x_2 . Two are her name (class 1), one is a clap (class 2), one is silence (class 3). "Three classes, two features, four examples. Batches of two," she says. "Let us do one full mini-batch step with nothing hidden." Ravi: "Every multiplication?" Asha: "Every one."

BIG IDEA

One mini-batch step is four matrix operations: $\mathbf{Z}_B = \mathbf{X}_B \mathbf{W}$, $\hat{\mathbf{Y}}_B = \text{softmax}(\mathbf{Z}_B)$ row by row, $\nabla J = \frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$, and $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla J$. Each entry of each matrix is a small sum of products you can do by hand. After training, using the model (**inference**) is just the first two operations plus arg max.

THINK OF IT LIKE...

A class register with three columns, one per subject. Each student (row) gets a mark in every subject from the same set of rules (the weight columns). Multiplying $\mathbf{X}_B \mathbf{W}$ fills in the whole register at once: row = student, column = subject. Softmax then converts each student's three marks into three shares. The gradient asks, subject by subject, "how far off were we, and which input caused it?"

Setup (slides 73–74)

x_1	x_2	Class
1	2	1
2	1	2
2	3	1
3	2	3

$K = 3$ classes, $d = 2$ features, $N = 4$ examples.

Batch size $B = 2$. Learning rate $\eta = 0.1$.

Write the data as matrices. The design matrix has a bias column of ones. The labels become one-hot rows. The weight matrix is $(d + 1) \times K = 3 \times 3$, initialised with small random numbers (your slide gives these):

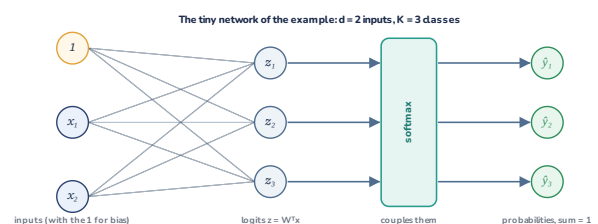


Figure 8.1 The network: inputs 1, x_1 , x_2 ; three logit neurons; softmax; three probabilities.

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 2 \end{bmatrix} \in \mathbb{R}^{4 \times 3}, \quad \mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 3}, \quad \mathbf{W}^{(0)} = \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix}$$

Read $\mathbf{W}^{(0)}$ by columns: column 1 is class 1's model, (0.1, 0.2, -0.1): bias 0.1, weight 0.2 on x_1 , weight -0.1 on x_2 .

WORKED EXAMPLE, PART 1: THE FORWARD PASS (SLIDES 75-76)

After shuffling, the first mini-batch is examples 1 and 2:

$$\mathbf{X}_B = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \mathbf{Y}_B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

STEP 1: LOGITS $\mathbf{Z}_B = \mathbf{X}_B \mathbf{W}^{(0)}$

Entry (row i , column k) is row i of $\mathbf{X}_B$ dotted with column k of $\mathbf{W}$. Row 1, all three classes:

- 1 $z_{1,1} = 1(0.1) + 1(0.2) + 2(-0.1) = 0.1 + 0.2 - 0.2 = 0.1$.
example 1 $\times$ class 1's column
- 2 $z_{1,2} = 1(-0.1) + 1(0.1) + 2(0.2) = -0.1 + 0.1 + 0.4 = 0.4$.
- 3 $z_{1,3} = 1(0.2) + 1(-0.1) + 2(0.1) = 0.2 - 0.1 + 0.2 = 0.3$.
- 4 Row 2 the same way: $z_{2,1} = 0.1 + 0.4 - 0.1 = 0.4$; $z_{2,2} = -0.1 + 0.2 + 0.2 = 0.3$; $z_{2,3} = 0.2 - 0.2 + 0.1 = 0.1$.

your slide prints 0.2 for $z_{2,2}$; the arithmetic gives 0.3, see the Careful box

$$\mathbf{Z}_B = \begin{bmatrix} 0.1 & 0.4 & 0.3 \\ 0.4 & 0.2 & 0.1 \end{bmatrix} \quad (\text{as printed on your slide})$$

$\mathbf{X}_B (2 \times 3)$	•	$\mathbf{W}^{(0)} (3 \times 3)$	=	$\mathbf{Z}_B (2 \times 3)$																						
<table border="1" style="border-collapse: collapse; width: 100px; height: 60px;"> <tr><td style="padding: 5px;">1</td><td style="padding: 5px;">1</td><td style="padding: 5px;">2</td></tr> <tr><td style="padding: 5px;">1</td><td style="padding: 5px;">2</td><td style="padding: 5px;">1</td></tr> </table>	1	1	2	1	2	1	•	<table border="1" style="border-collapse: collapse; width: 150px; height: 100px;"> <tr><td style="padding: 5px;">0.1</td><td style="padding: 5px;">-0.1</td><td style="padding: 5px;">0.2</td></tr> <tr><td style="padding: 5px;">0.2</td><td style="padding: 5px;">0.1</td><td style="padding: 5px;">-0.1</td></tr> <tr><td style="padding: 5px;">-0.1</td><td style="padding: 5px;">0.2</td><td style="padding: 5px;">0.1</td></tr> </table>	0.1	-0.1	0.2	0.2	0.1	-0.1	-0.1	0.2	0.1	=	<table border="1" style="border-collapse: collapse; width: 100px; height: 60px;"> <tr><td style="padding: 5px;">0.1</td><td style="padding: 5px;">0.4</td><td style="padding: 5px;">0.3</td></tr> <tr><td style="padding: 5px;">0.4</td><td style="padding: 5px;">0.3</td><td style="padding: 5px;">0.1</td></tr> </table>	0.1	0.4	0.3	0.4	0.3	0.1	row 1 of $\mathbf{X}_B$ • column 1 of $\mathbf{W}$: $1 \cdot 0.1 + 1 \cdot 0.2 + 2 \cdot (-0.1)$ $= 0.1 + 0.2 - 0.2 = 0.1$
1	1	2																								
1	2	1																								
0.1	-0.1	0.2																								
0.2	0.1	-0.1																								
-0.1	0.2	0.1																								
0.1	0.4	0.3																								
0.4	0.3	0.1																								

Each entry of $\mathbf{Z}_B$ = (a row of $\mathbf{X}_B$) • (a column of $\mathbf{W}$). Row = which example, column = which class.

The bias column of 1s in $\mathbf{X}_B$ multiplies row 0 of $\mathbf{W}$, so the biases are added automatically.

STEP 2: SOFTMAX, ROW BY ROW

- 1 Row 1: $e^{0.1} = 1.105$, $e^{0.4} = 1.492$, $e^{0.3} = 1.350$; sum = 3.947.
- 2 $\hat{\mathbf{y}}^{(1)} = (1.105, 1.492, 1.350)/3.947 = (0.280, 0.378, 0.342)$.
the model currently favours class 2 for a class-1 example
- 3 Row 2, from your slide: $\hat{\mathbf{y}}^{(2)} = (0.387, 0.315, 0.298)$.
we keep the slide's row so the rest matches your slides digit for digit

$$\hat{\mathbf{Y}}_B = \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix}$$

WORKED EXAMPLE, PART 2: GRADIENT AND UPDATE (SLIDES 77–78)

STEP 3: GRADIENT $\nabla J = \frac{1}{B} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$

1 Error matrix, entry by entry: $\hat{\mathbf{Y}}_B - \mathbf{Y}_B = \begin{bmatrix} 0.280 - 1 & 0.378 - 0 & 0.342 - 0 \\ 0.387 - 0 & 0.315 - 1 & 0.298 - 0 \end{bmatrix} = \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix}$.

negative where the true class was under-predicted, positive elsewhere

2 $\mathbf{X}_B^\top = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \end{bmatrix}$, a 3×2 matrix; times the 2×3 error gives 3×3 ; then divide by $B = 2$.

3 Row 0 is the bias row; both inputs are 1, so just add the two rows of the error: $(-0.720 + 0.387, 0.378 - 0.685, 0.342 + 0.298) = (-0.333, -0.307, 0.640)$.

4 Divide by $B = 2$: row 0 of $\nabla J = (-0.167, -0.154, 0.320)$.

5 Rows 1 and 2 the same way (row 1 uses inputs 1, 2; row 2 uses 2, 1). Your slide's result:

$$\nabla J \approx \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix}$$

STEP 4: UPDATE $\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - H \nabla J$

1 Entry (0,1): $0.1 - 0.1 \times (-0.167) = 0.1 + 0.0167 = 0.117$.

a negative gradient means "increase this weight"

2 Entry (0,2): $-0.1 - 0.1 \times (-0.154) = -0.1 + 0.0154 = -0.085$.

3 Entry (0,3): $0.2 - 0.1 \times 0.320 = 0.2 - 0.032 = 0.168$.

class 3 was over-predicted for both examples, so its bias goes down

4 All nine entries:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.117 & -0.085 & 0.168 \\ 0.202 & 0.150 & -0.147 \\ -0.047 & 0.196 & 0.051 \end{bmatrix}$$

This completes one iteration of mini-batch SGD. Next: process the second mini-batch (examples 3 and 4) with $\mathbf{W}^{(1)}$, and continue for multiple epochs until convergence.

! CAREFUL: THREE SMALL SLIPS ON SLIDES 75–77

If you redo the arithmetic you will find three places where the printed numbers do not follow from the ones before. (a) $z_{2,2} = -0.1 + 0.2 + 0.2 = 0.3$, but the slide prints 0.2. (b) The softmax of the correct row 2, $(0.4, 0.3, 0.1)$, is $(0.378, 0.342, 0.280)$; the slide prints $(0.387, 0.315, 0.298)$. (c) Using the slide's own error matrix, gradient entry (1,1) is $(1 \times (-0.720) + 2 \times 0.387)/2 = +0.027$, not -0.024 . Carried through exactly, the true update is $\mathbf{W}^{(1)} = [0.117, -0.086, 0.169; 0.198, 0.147, -0.145; -0.047, 0.195, 0.052]$; the same to two decimals. The *method* is what you must know; we kept the slide's printed values so you can follow them in the exam.

DOES IT LEARN?

We kept going in `verify.py`, two mini-batches per epoch in fixed order. After 1 epoch the model predicts class 1 for every point. After 10 epochs it gets three of the four right. After 200 epochs it classifies all four correctly, giving the true class probabilities 0.967, 0.643, 0.948 and 0.786. Slow, because $\eta = 0.1$ is small and the points are close together, but it gets there.

The computational graph

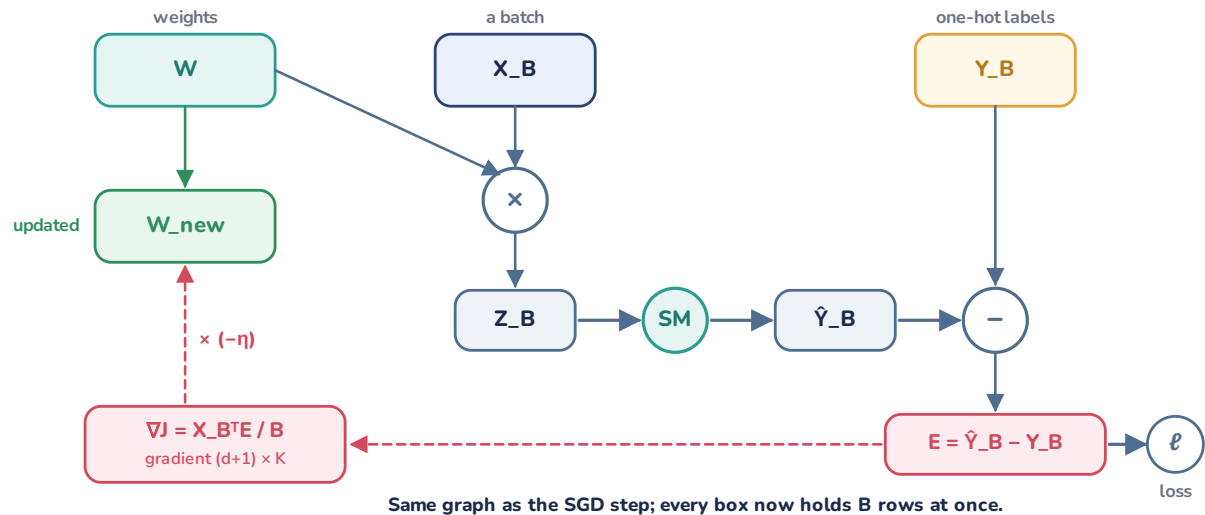


Figure 8.2 Your slide 79. It is Figure 4.4 with every box widened to hold B rows. X_B replaces one x ; Z_B replaces z ; softmax (SM) replaces σ ; an error matrix E replaces a number. The backward path is $\frac{1}{B} X_B^T E$. Notice the caption on your slide: B examples are processed at the same time, efficient and stable.

Inference: using the trained model (slides 81–82)

Training is over; W is fixed. A new input x_{new} arrives (with its leading 1). Three steps, equations (28)–(30):

$$z = W^T x_{\text{new}}, \quad \hat{y} = \text{softmax}(z), \quad \hat{k} = \arg \max_{k=1, \dots, K} \hat{y}_k$$

Optionally, report the probabilities $\hat{y}$ as confidence scores. No labels, no loss, no gradient: inference is the forward pass only.

WORKED EXAMPLE: INFERENCE (SLIDE 82)

Trained model, $K = 3$, new input $x = [1, 2.5, 1.8]^T$.

- 1 Logits: $z = W^T x = [1.2, 2.5, 0.8]^T$.
your slide gives the result of this multiplication
- 2 $e^{1.2} \approx 3.32$, $e^{2.5} \approx 12.18$, $e^{0.8} \approx 2.23$.
- 3 Sum = $3.32 + 12.18 + 2.23 = 17.73$.
- 4 $\hat{y} = (3.32, 12.18, 2.23) / 17.73 = (0.187, 0.687, 0.126)$.
- 5 $\hat{k} = \arg \max_k \hat{y}_k = 2$.
class 2 with 68.7% confidence

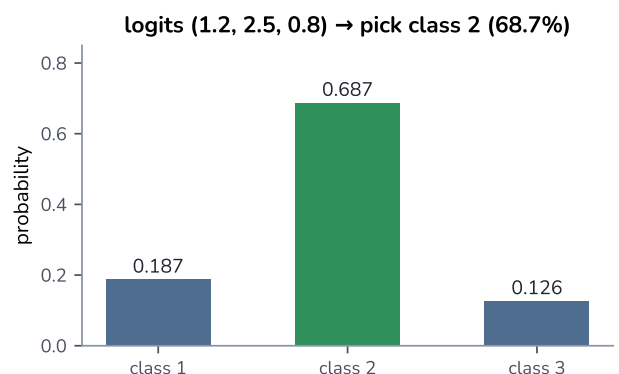


Figure 8.3 The three probabilities. Class 2 wins clearly, but the model still gives 31% to the other two. That is useful information a hard "class 2" answer would hide.

? STOP AND THINK

Ravi: "In step 1 of the mini-batch, why does the gradient's bias row get computed with inputs 1 and 1?"

✓ OUR ANSWER

The bias column of $\mathbf{X}_B$ is all ones, so row 0 of $\mathbf{X}_B^\top$ is $(1, 1)$. Multiplying by the error matrix simply *adds* the two examples' errors, class by class, then divides by B . The bias gradient is the average error for each class. Rows 1 and 2 weight the errors by x_1 and x_2 instead.

☰ ON YOUR SLIDES

Slide 73: the four points, the network, $K = 3, d = 2, N = 4, B = 2, \eta = 0.1$. Slide 74: $\mathbf{X}$, one-hot $\mathbf{Y}$, $\mathbf{W}^{(0)}$. Slide 75: $\mathbf{X}_B, \mathbf{Y}_B, \mathbf{Z}_B$. Slide 76: the softmax rows. Slide 77: $\hat{\mathbf{Y}}_B - \mathbf{Y}_B$ and ∇J . Slide 78: $\mathbf{W}^{(1)}$ and "this completes one iteration". Slide 79: the computational graph. Slide 81: equations (28)–(30). Slide 82: the inference example, class 2 with 68.7% confidence.

✓ CHECK YOURSELF

1. What is the shape of $\mathbf{Z}_B$ here, and why? 2×3 : $B = 2$ examples, $K = 3$ classes.
2. Compute $z_{2,1}$ yourself from $\mathbf{X}_B$ and $\mathbf{W}^{(0)}$. $1(0.1) + 2(0.2) + 1(-0.1) = 0.4$.
3. Why is entry (1,1) of $\hat{\mathbf{Y}}_B - \mathbf{Y}_B$ negative? **Example 1 is class 1; the model gave it only 0.280, less than the target 1.**
4. Which of the three inference steps needs the labels? **None. Inference uses no labels.**

🔴 IN ONE BREATH

- $\mathbf{X}_B$ (2×3) times $\mathbf{W}^{(0)}$ (3×3) gives $\mathbf{Z}_B$ (2×3); row 1 is $(0.1, 0.4, 0.3)$.
- Softmax each row: $(0.280, 0.378, 0.342)$ and $(0.387, 0.315, 0.298)$. Subtract the one-hot rows to get the error.
- $\nabla J = \frac{1}{2} \mathbf{X}_B^\top (\hat{\mathbf{Y}}_B - \mathbf{Y}_B)$; $\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - 0.1 \nabla J$, first row $(0.117, -0.085, 0.168)$.
- Inference: $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$, softmax, argmax. $(1.2, 2.5, 0.8) \rightarrow (0.187, 0.687, 0.126) \rightarrow \text{class 2}$.

Marking a many-class machine

A $K \times K$ confusion matrix, precision and recall per class, two ways to average them, and top- K accuracy.

On your slides: **Module 4, slides 84–89** · Evaluation metrics for multi-class · Confusion matrix for multi-class · Per-class metrics · Averaging strategies · Top- K accuracy · Summary

STORY

Asha tests the three-sound Nova on 150 new clips, 50 of each kind. It gets 127 right. "84.7%," says Ravi. "Which class does it get wrong?" Asha draws a 3-by-3 table. Claps are the problem: 7 were called "silence", 5 were called "Nova". "Worst at claps," says Ravi. "Give me a number for that."

BIG IDEA

With K classes the confusion matrix becomes $K \times K$: row = true class, column = predicted class. Accuracy is the diagonal divided by everything. Precision and recall are computed *one class at a time*, treating that class as "positive" and all others as "negative", then averaged, either plainly (macro) or weighted by class size.

THINK OF IT LIKE...

A sorting office with three pigeon-holes. The confusion matrix is the audit: for every letter, which hole should it have gone in (row) and which hole did it land in (column)? The diagonal is the letters that arrived correctly. To judge one hole alone: of the letters in it, how many belong (precision)? Of the letters that belong, how many arrived (recall)?

Accuracy and the $K \times K$ confusion matrix

		Predicted			
		Class 1	Class 2	Class 3	row sum
Actual	Class 1	45	3	2	= 50
	Class 2	5	38	7	= 50
	Class 3	2	4	44	= 50
column sum		= 52	= 45	= 53	

diagonal (green) = right; off-diagonal (red) = wrong. Accuracy = $(45 + 38 + 44) / 150 = 0.847$

Figure 9.1 Your slide 85. Row i , column j holds the number of class- i examples that were predicted as class j . Green diagonal: correct. Red off-diagonal: mistakes. Each row sums to 50, the number of actual examples per class. The column sums (52, 45, 53) are how many times each class was *predicted*.

Accuracy is the same idea as before, written with an **indicator**, equation (31):

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{N} = \frac{1}{N} \sum_{i=1}^N 1[\hat{y}^{(i)} = y^{(i)}]$$

The symbol $1[\cdot]$ is 1 when the statement inside is true and 0 otherwise. So the sum simply counts the examples whose predicted class equals the true class.

Accuracy from the matrix: $(45 + 38 + 44)/150 = 127/150 = 0.847$, or 84.7%. Reading the red cells tells you *which* confusions happen: 7 class-2 examples were called class 3, the biggest single mistake.

Per-class precision, recall and F1

For each class k , pretend the problem is binary: class k versus everything else ("one-vs-all"). Then TP_k is the diagonal cell. FP_k is the rest of column k (predicted k , but not k). FN_k is the rest of row k (truly k , but predicted otherwise). Equations (32)–(34):

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = 2 \cdot \frac{\text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$$

WORKED EXAMPLE: ALL THREE CLASSES FROM FIGURE 9.1

- 1 Class 1: $TP = 45$; $FP = 5 + 2 = 7$ (rest of column 1); $FN = 3 + 2 = 5$ (rest of row 1).
- 2 $\text{Precision}_1 = 45/52 = 0.865$; $\text{Recall}_1 = 45/50 = 0.900$; $F1_1 = 2 \times \frac{0.865 \times 0.900}{1.765} = 0.882$.
- 3 Class 2: $TP = 38$; $FP = 3 + 4 = 7$; $FN = 5 + 7 = 12$.
- 4 $\text{Precision}_2 = 38/45 = 0.844$; $\text{Recall}_2 = 38/50 = 0.760$; $F1_2 = 0.800$.
the low recall is Ravi's "worst at claps" number: 12 claps were missed
- 5 Class 3: $TP = 44$; $FP = 2 + 7 = 9$; $FN = 2 + 4 = 6$.
- 6 $\text{Precision}_3 = 44/53 = 0.830$; $\text{Recall}_3 = 44/50 = 0.880$; $F1_3 = 0.854$.

Class	TP	FP	FN	Precision	Recall	F1
1	45	7	5	0.865	0.900	0.882
2	38	7	12	0.844	0.760	0.800
3	44	9	6	0.830	0.880	0.854

The per-class numbers are ours (the slide gives only the formulas); check them against `verify.py`.

Averaging: macro or weighted

Three precisions is three numbers. To get one overall score, your slide 87 gives two recipes.

MACRO-AVERAGE, EQUATION (35)

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k$$

A plain average: every class counts the same, big or small. Good when all classes are equally important.

Here: $(0.865 + 0.844 + 0.830)/3 = 0.847$.

→

WEIGHTED AVERAGE, EQUATION (36)

$$\text{Precision}_{\text{weighted}} = \sum_{k=1}^K w_k \text{Precision}_k, \quad w_k = \frac{N_k}{N}$$

N_k is the number of examples of class k .

Common classes count more. Accounts for class imbalance; good when some classes are more common.

Here all $N_k = 50$, so $w_k = 1/3$ and the weighted average is also 0.847. With classes of size 90 and 10 and precisions 0.9 and 0.5, macro gives 0.70 but weighted gives 0.86.

The same two recipes work for recall and F1. In our example, macro recall is $(0.900 + 0.760 + 0.880)/3 = 0.847$ and macro F1 is 0.846.

Top-K accuracy

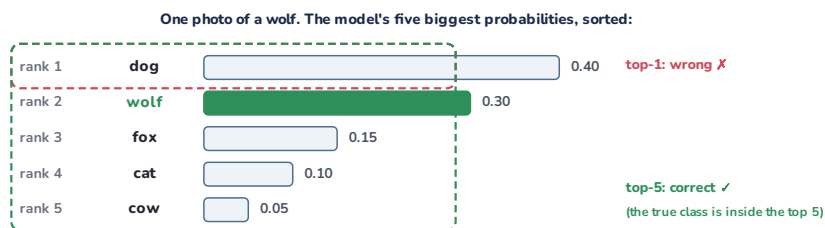


Figure 9.2 A photo of a wolf, and the five classes the model ranked highest. The top-1 guess ("dog") is wrong. But "wolf" is at rank 2, so the top-5 guess is counted as correct.

With 1000 classes, as in ImageNet, demanding the exact top answer is harsh: "husky" for a "wolf" is close. **Top-K accuracy** counts a prediction as correct if the true class is anywhere in the model's K highest probabilities, equation (37):

$$\text{Top-}K \text{ Accuracy} = \frac{1}{N} \sum_{i=1}^N 1[y^{(i)} \in \text{Top-}K(\hat{y}^{(i)})]$$

Top-1: is the highest-probability class correct? Top-5: is the true class among the five highest? Why useful: more forgiving for large K ; it captures whether the right class is among the top candidates; and it is the standard in the ImageNet challenge. (The K in "top- K " is a small number like 5, not the number of classes.)

Summary of multi-class classification

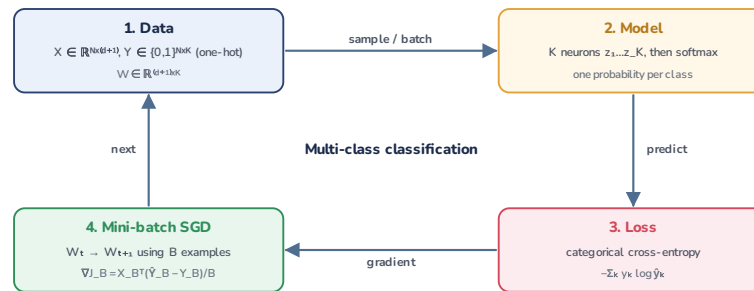


Figure 9.3 Your slide 89: the loop again. Data $\mathbf{X}$ and one-hot $\mathbf{Y}$, weights $\mathbf{W}$; a batch goes to K neurons plus softmax; cross-entropy scores it; mini-batch SGD moves $\mathbf{W}_t$ to $\mathbf{W}_{t+1}$; iterate.

? STOP AND THINK

Ravi: "Macro and weighted gave the same answer. When would I ever see a difference?"

✓ OUR ANSWER

Whenever the classes are unequal in size. Imagine 990 "silence" clips and 10 "Nova" clips. A model that is great on silence and terrible on "Nova" has a high weighted score (silence dominates) but a poor macro score (each class counts equally). If the rare class is the one you care about, look at macro, or at that class's own recall.

📖 ON YOUR SLIDES

Slide 84: accuracy with the indicator, (31); the $K \times K$ matrix; per-class metrics, one-vs-all, macro or weighted. Slide 85: the 45/38/44 matrix, row sums 50, accuracy 0.847. Slide 86: (32)–(34). Slide 87: (35)–(36). Slide 88: (37), top-1 and top-5, ImageNet. Slide 89: the summary loop.

! CAREFUL

FP_k comes from the *column* (things wrongly called k); FN_k from the *row* (real k 's that went elsewhere). "Predicted k " is a column, "actually k " is a row.

✓ CHECK YOURSELF

1. In Figure 9.1, how many class-3 examples were predicted as class 2? **4 (row 3, column 2).**
2. Which average treats every class equally? **Macro.**
3. A model ranks the true class 3rd. Correct under top-1? Under top-5? **No; yes.**

🔥 IN ONE BREATH

- Accuracy = $\frac{1}{N} \sum 1[\hat{y} = y]$; from the matrix, diagonal over total: $127/150 = 0.847$.
- Per class: TP_k on the diagonal, FP_k rest of the column, FN_k rest of the row; precision, recall, F1 as before.
- Macro = plain average over classes; weighted = weighted by N_k/N .
- Top-K accuracy: correct if the true class is in the K best guesses. Top-1 and top-5 for ImageNet.

Tips, debugging, and the big comparison

What to set, what to check when it breaks, and one table that lines up regression, binary and multi-class side by side.

On your slides: **Module 4, slides 91–99** · Implementation tips · Debugging checklist · Activation functions · Loss functions · Gradient descent variants · Comparison of linear models · Recap: binary · Recap: multi-class

📖 STORY

Nova now hears its name, and can tell a clap from silence. Asha writes a checklist on the wall so she never again spends a night on a bug that was just a learning rate. Then she tries something harder: a recording where "Nova" is said softly at a high pitch or loudly at a low pitch, but not in between. The three-sound model cannot learn it, however long it trains. Ravi: "It is the XOR problem from Book 2 again." Asha: "A straight line still cannot bend."

! BIG IDEA

Regression, binary classification and multi-class classification are *one* model, a linear neuron layer, with three different output activations (identity, sigmoid, softmax) and three matching losses (MSE, binary CE, categorical CE). Get the practical settings right, learn to read the symptoms when training misbehaves, and you can train any of the three.

≈ THINK OF IT LIKE...

A pressure cooker with a recipe card taped to it. The card says how much heat (learning rate), how long (epochs), how to prepare the vegetables (scaling), and what each hissing sound means (the debugging list). The cooker is the same whether you make dal, rice or sambar; only the card changes. This chapter is the card.

Implementation tips (slide 91)

1. Learning rate and epochs

Start with $\eta \in \{0.01, 0.1, 1.0\}$ and see which works. Typical: 10–100 epochs. Use **early stopping**: stop when the validation loss stops improving.

2. Weight initialisation

$w_j \sim \mathcal{N}(0, 0.01)$, small random values. Avoid all zeros (every class starts identical) and large values (saturation).

3. Batch size

Start with $B \in \{64, 128, 256\}$. Powers of two preferred. Larger: stable but slower. Smaller: noisy but faster.

4. Data handling

Always shuffle each epoch. Check class balance: if one class dominates, accuracy lies and the model may just predict that class.

5. Feature scaling: essential

Standardise each feature: $x'_j = (x_j - \mu_j) / \sigma_j$, subtract the mean, divide by the standard deviation. This prevents sigmoid saturation.

6. Numerical stability

Use a stable formula for $\log \sigma(z)$. Use the stable softmax with $c = \max_k z_k$. Use the log-sum-exp trick. All prevent overflow and underflow.

Two of these deserve a closer look. **Saturation** means the sigmoid is out on its flat ends. Suppose hours were measured in minutes (60, 120, 180, 240). Then z would be huge, $\sigma(z)$ would sit at 0.99995, and the slope $\sigma'(z) = 0.00005$ would make learning crawl. Standardising fixes it.

Σ MATHS, STEP BY STEP

PLAIN MATHS: STANDARDISE ONE FEATURE

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

μ_j is the mean of feature j over the training set, σ_j its standard deviation. The study hours (1, 2, 3, 4) have mean 2.5 and standard deviation 1.118, so they become $(-1.342, -0.447, 0.447, 1.342)$: small numbers around zero, where the sigmoid is steep.

STABLE SOFTMAX, ONE ROW OF LOGITS

$$\text{SM}(\mathbf{z})_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}, \quad c = \max_k z_k$$

Subtracting c from every logit leaves the result unchanged (softmax property 4) but keeps every $e^{z_k - c} \leq 1$. For a batch, subtract each row's own maximum. The **log-sum-exp** trick is the same idea for $\log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}$, which appears inside the cross-entropy loss.

naive

$\mathbf{z} = (1000, 1001, 999)$

$e^{1000} = \text{overflow } (\infty)$

$\infty / \infty = \text{NaN}$

the computer cannot hold e^{1000} — it gives up

stable

$c = \max \mathbf{z} = 1001$

$\mathbf{z} - c = (-1, 0, -2)$

$e^{\mathbf{z} - c} = (0.368, 1, 0.135)$

$\div 1.503 \rightarrow (0.245, 0.665, 0.090)$

subtracting the same c from every logit does not change the answer (property 4), but keeps every $e^z \leq 1$

log-sum-exp uses the same trick: $\log \sum e^z = c + \log \sum e^{z - c}$

Figure 10.1 Why the stable softmax exists. Top: logits around 1000 make e^z overflow, and the computer returns NaN ("not a number"). Bottom: subtract $c = \max_k z_k$ first, and every number stays small. Same answer, no overflow.

Debugging checklist (slide 92)

Symptom	What to check, in order
Loss is NaN or Inf	η too large $\rightarrow$ decrease it. A $\log(0)$ in the loss. Unscaled features. Use the stable softmax.
Predicts the same class for everything	Class balance. The softmax and loss code. η might be too small. The one-hot encoding.
Loss oscillates	η too large. Use mini-batches instead of single examples. Apply learning-rate decay (shrink η over time).
Softmax outputs do not sum to 1	An implementation bug (normalising the wrong axis). Numerical precision. Use the stable softmax.
Loss not decreasing	Gradient signs. η too small or too large. Feature scaling. Data shuffling. One-hot encoding.
High train accuracy, low test accuracy	Overfitting: add regularisation, get more training data, or use a simpler model.
Sanity checks	A tiny dataset should reach 100% accuracy. Verify $\hat{y} \in [0, 1]$ and $\sum_k \hat{y}_k = 1$. Shuffling should improve results. Print the confusion matrix.

The three comparison tables (slides 94–96)

ACTIVATION FUNCTIONS

Property	Identity	Sigmoid	Softmax
Formula	$f(z) = z$	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\text{SM}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Output range	$(-\infty, \infty)$	$[0, 1]$	$[0, 1]^K, \sum_k = 1$
Use case	regression	binary classification	multi-class
Number of outputs	1	1	K (coupled)

LOSS FUNCTIONS

Aspect	MSE	Binary CE	Categorical CE
Formula	$\frac{1}{N} \sum_i (\hat{y}^{(i)} - y^{(i)})^2$	$-\frac{1}{N} \sum_i [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$	$-\frac{1}{N} \sum_i \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$
Task	regression	binary classification	multi-class classification
Output type	continuous value	probability (2 classes)	probability distribution (K classes)
Target y	real number	$\{0, 1\}$	one-hot vector
Prediction $\hat{y}$	any real number	probability in $[0, 1]$	probability vector, $\sum_k \hat{y}_k = 1$
Activation	identity	sigmoid	softmax
Penalises	distance from target	confident wrong predictions	wrong-class probability
Properties	convex, quadratic penalty	convex, probabilistic, logarithmic penalty	convex, extends binary CE, multi-class penalty

GRADIENT DESCENT VARIANTS

Aspect	Batch GD	Stochastic GD	Mini-batch GD
Examples per update	all N	1	B (batch size)
Gradient	$\nabla J = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i$	$\nabla \ell_i$ for a random i	$\nabla J_B = \frac{1}{B} \sum_{i \in B} \nabla \ell_i$
Update rule	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J_B$
Iterations per epoch	1	N	$\lceil N/B \rceil$
Speed per iteration	slow	fast	medium
Convergence	smooth, deterministic	noisy, oscillating	balanced
Memory	high (all data)	low (1 example)	medium (B examples)
Gradient quality	exact	noisy estimate	good estimate
Advantages	stable, precise gradient, reproducible	fast iterations, online learning, escapes local minima	best balance, GPU efficient, stable and fast
Disadvantages	slow for large N , high memory, no online learning	very noisy, unstable, hard to parallelise	extra hyperparameter B , needs tuning

Aspect	Batch GD	Stochastic GD	Mini-batch GD
Best for	small datasets	online learning	deep learning (standard)
Typical B	N (all data)	1	32, 64, 128, 256

Comparison of linear models (slide 97)

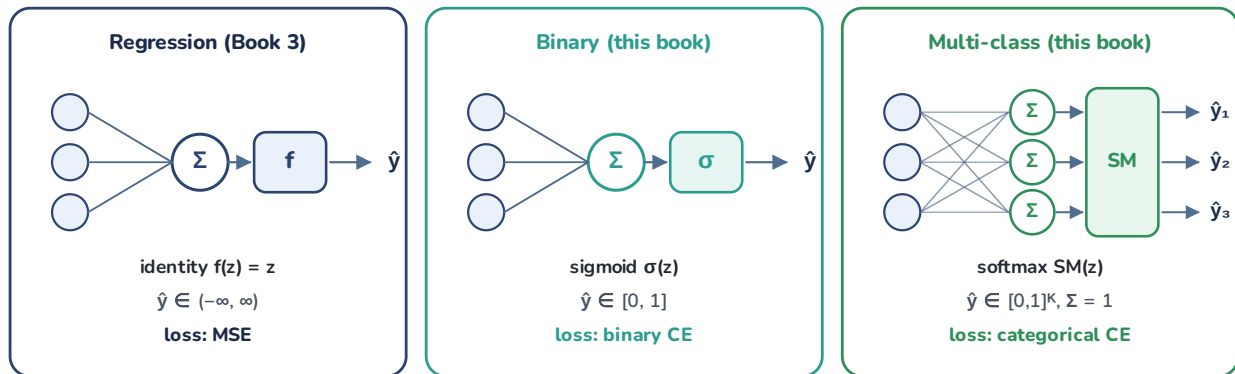


Figure 10.2 The three linear models of Books 3 and 4 side by side. Same inputs, same weighted sums. Only the box after the sum changes, and with it the loss. Notice the multi-class model is just K copies of the sum, tied together by one softmax.

Aspect	Regression	Binary	Multi-class
Output	continuous $y \in \mathbb{R}$	discrete $y \in \{0, 1\}$	discrete $y \in \{1, \dots, K\}$
Activation	identity $f(z) = z$	sigmoid $\sigma(z)$	softmax $SM(z)$
Loss function	mean squared error	binary cross-entropy	categorical cross-entropy
Output range	$(-\infty, \infty)$	$[0, 1]$	$[0, 1]^K, \sum_k = 1$
Interpretation	predicted value	$P(y = 1 \mid \mathbf{x})$	$P(y = k \mid \mathbf{x})$ for all k
Output neurons	1	1	K
Weights	vector $\mathbf{w} \in \mathbb{R}^{d+1}$	vector $\mathbf{w} \in \mathbb{R}^{d+1}$	matrix $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$
Evaluation	MSE, RMSE, R^2	accuracy, precision, recall, F1	confusion matrix, top-K
Learning	batch GD or SGD	SGD or mini-batch	mini-batch SGD
Examples	house prices, temperature	spam detection, diagnosis	digit recognition, ImageNet

Recap (slides 98–99)

Binary classification: logistic regression

Key insight: the sigmoid maps a linear combination to a probability; train by minimising cross-entropy. Problem: two classes, $y \in \{0, 1\}$. Model: one neuron with sigmoid. Output: $\hat{y} = P(y = 1 \mid \mathbf{x}) \in [0, 1]$, then class 0 or 1. Loss: binary cross-entropy. Training: SGD.

Multi-class classification

Key insight: multiple outputs coupled through softmax form a valid probability distribution over all classes. Problem: K classes, $y \in \{1, \dots, K\}$. Model: K output neurons with softmax. Output: $\hat{\mathbf{y}} = [\hat{y}_1, \dots, \hat{y}_K]^\top, \sum_k \hat{y}_k = 1$. Loss: categorical cross-entropy. Training: mini-batch SGD.

! CAREFUL

Every model in this book draws a *straight* decision boundary (a hyperplane). Softmax with K classes draws several straight boundaries, but each one is still straight. No amount of training changes that.

What Nova still cannot do

Asha's soft-high-or-loud-low recording is the XOR pattern from Book 2: the "Nova" points sit in two opposite corners, the others in the remaining two. No single straight line separates them, and neither do the straight boundaries of a softmax layer.

What would help is a boundary that *bends*. To bend, we need to stack neurons: a first layer that draws several lines, and a second layer that combines them. That is a hidden layer, and it is the whole of Book 5, *A Real Brain*.

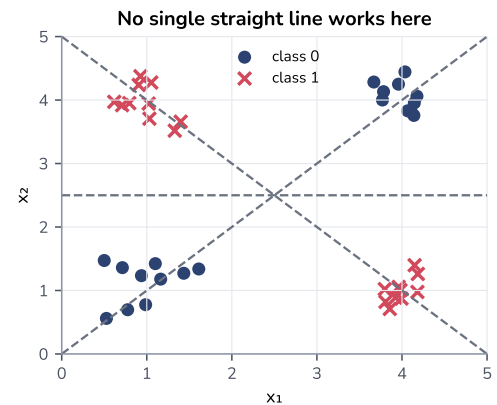


Figure 10.3 Two classes in opposite corners. Every dashed straight line leaves a class split. Layers, at last.

📄 ON YOUR SLIDES

Slide 91: the six tip groups, with $x'_j = (x_j - \mu_j)/\sigma_j$ and the stable softmax. Slide 92: the debugging checklist. Slides 94–96: the three tables. Slide 97: comparison of linear models. Slides 98–99: the two recaps with their neuron pictures.

✅ CHECK YOURSELF

1. The loss becomes NaN on the first step. First suspect? **Learning rate too large (then log 0, unscaled features, unstable softmax).**
2. Why subtract $\max_k z_k$ before the softmax? **It leaves the answer unchanged and stops e^z from overflowing.**
3. Train accuracy 99%, test accuracy 60%. What is wrong? **Overfitting: regularise, get more data, or simplify the model.**

🧠 IN ONE BREATH

- Settings: $\eta \in \{0.01, 0.1, 1\}$, 10–100 epochs, early stopping, $\mathcal{N}(0, 0.01)$ init, $B \in \{64, 128, 256\}$, shuffle, standardise. Stability: subtract the max before softmax; log-sum-exp.
- Debug by symptom: NaN $\rightarrow \eta$; same class $\rightarrow$ balance; oscillation $\rightarrow \eta$ or mini-batches; train $\gg$ test $\rightarrow$ overfitting.
- One linear model, three heads: identity + MSE, sigmoid + binary CE, softmax + categorical CE. All draw straight boundaries. Book 5 bends them.

Where you are in the series

Six small books, one story. Asha builds Nova, a robot that must learn to hear its name — and along the way she meets every idea in the Deep Neural Network course, in the order the course teaches them.

BOOK 1

Teaching a Machine to Listen

What deep learning is, and the four parts of every learning problem

BOOK 2

One Tiny Brain Cell

The artificial neuron, the perceptron, logic gates and the XOR wall

BOOK 3

Guessing How Much

A linear neuron for regression, squared loss and gradient descent

BOOK 4

Guessing Which One

Sigmoid and softmax neurons, cross-entropy, SGD and mini-batches

BOOK 5

A Real Brain

Hidden layers, activations, backpropagation by hand, and design

BOOK 6

Eyes for the Machine

Convolution, feature maps, pooling, famous CNNs and transfer learning

Next: **Book 5, A Real Brain** — hidden layers, the activation functions that let a network bend, and backpropagation done fully by hand.